

Code as Agent Harness

◇ TOWARD EXECUTABLE, VERIFIABLE, AND STATEFUL AGENT SYSTEMS ◇

Xuying Ning^{1†} Katherine Tieu^{1†} Dongqi Fu^{2†} Tianxin Wei^{1†} Zihao Li^{1†} Yuanchen Bei^{1†}
 Jiaru Zou³ Mengting Ai¹ Zhining Liu¹ Ting-Wei Li¹ Lingjie Chen¹ Yanjun Zhao¹ Ke Yang¹
 Bingxuan Li¹ Cheng Qian¹ Gaotang Li¹ Xiao Lin¹ Zhichen Zeng¹ Ruizhong Qiu¹ Sirui Chen¹
 Yifan Sun¹ Xiyuan Yang¹ Ruida Wang¹ Rui Pan¹ Chenyuan Yang¹ Dylan Zhang¹ Liri Fang¹
 Zikun Cui² Yang Cao² Pan Chen² Dorothy Sun² Ren Chen²
 Mahesh Srinivasan² Nipun Mathur² Yinglong Xia² Hong Li² Hong Yan²
 Pan Lu³ Lingming Zhang¹ Tong Zhang¹ Hanghang Tong^{1✉} Jingrui He^{1✉}

¹University of Illinois Urbana-Champaign ²Meta ³Stanford University

[†] Core Contributor, ✉ Corresponding Author

Abstract: Recent large language models (LLMs) have demonstrated strong capabilities in understanding and generating code, from competitive programming to repository-level software engineering. In emerging agentic systems, code is no longer only a target output. It increasingly serves as an operational substrate for agent reasoning, acting, environment modeling, and execution-based verification. We frame this shift through the lens of *agent harnesses* and introduce *code as agent harness*: a unified view that centers code as the basis for agent infrastructure. To systematically study this perspective, we organize the survey around three connected layers. First, we study the *harness interface*, where code connects agents to reasoning, action, and environment modeling. Second, we examine *harness mechanisms*: planning, memory, and tool use for long-horizon execution, together with feedback-driven control and optimization that make harness reliable and adaptive. Third, we discuss *scaling the harness* from single-agent systems to multi-agent settings, where shared code artifacts support multi-agent coordination, review, and verification. Across these layers, we summarize representative methods and practical applications of *code as agent harness*, spanning coding assistants, GUI/OS automation, embodied agents, scientific discovery, personalization and recommendation, DevOps, and enterprise workflows. We further outline open challenges for harness engineering, including evaluation beyond final task success, verification under incomplete feedback, regression-free harness improvement, consistent shared state across multiple agents, human oversight for safety-critical actions, and extensions to multimodal environments. By centering code as the harness of agentic AI, this survey provides a unified roadmap toward executable, verifiable, and stateful AI agent systems.

🔍 **Keywords:** Agent Harness, Coding Agent, Harness Engineering, Agentic AI

🔗 **Github:** <https://github.com/YennNing/Awesome-Code-as-Agent-Harness-Papers>

1. Introduction

Recent large language models (LLMs) have demonstrated strong capabilities in understanding and generating code [1, 2, 3], achieving strong performance in tasks ranging from competitive programming [4] to repository-level software engineering [5]. Building on these capabilities, the role of code in agentic systems is expanding beyond a target artifact to be generated. Programs are increasingly used as the medium through which

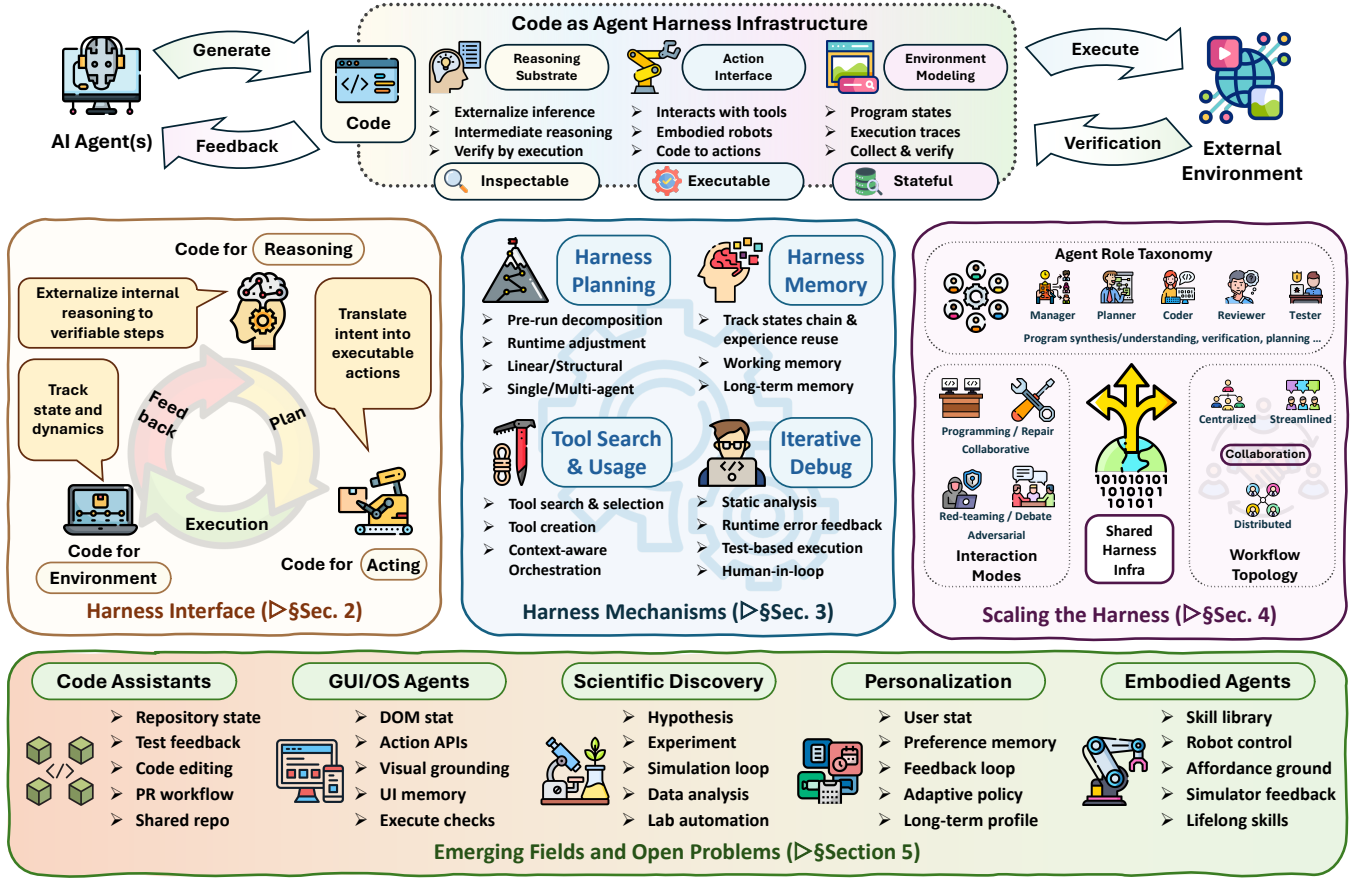


Figure 1: Taxonomy of code as agent harness.

LLM agents reason, act, and model their environments. Program-aided reasoning methods externalize intermediate computation into executable code [6, 7, 8]; robotic and embodied agents use generated programs as executable policies for interacting with physical or simulated worlds [9, 10]; and software-engineering or interactive environments use codebases, execution traces, tests, and runtime feedback as structured representations of environment state and dynamics, in which agents plan, act, and revise their behavior [11, 5, 12]. Taken together, these developments suggest a broader view: code is not only an artifact generated by LLMs, but also an executable, inspectable, and stateful medium through which agents reason, act, observe feedback, and verify progress. We refer to this view as *code as agent harness*.

Recent discussions on *agent harnesses* [13, 14, 15, 16] provide a useful system-level lens for understanding this shift. An agent harness refers to the software layer that surrounds an LLM with tools, APIs, sandboxes, memory, validators, permission boundaries, execution loops, and feedback channels, thereby turning a stateless model into a functional agent capable of long-running task execution [17, 18, 19, 20, 21, 22, 23]. In this view, the bottleneck of autonomy is not only the reasoning ability of the base model, but also the reliability of the system that connects model outputs to long-horizon actions and persistent states.

To clarify the role of code in this broader harness view, we distinguish three coupled elements of long-running agentic systems: *model-internal capabilities*, *system-provided harness infrastructure*, and *agent-initiated code artifacts*. *Model-internal capabilities* refer to the model’s reasoning, perception, planning, simulation, and evaluation abilities. *System-provided harness infrastructure* refers to the predefined tools, APIs, sandboxes, memory systems, validators, permission boundaries, telemetry, and workflows that connect model outputs

to external actions and feedback, and forms the main focus of harness engineering [24, 25]. In contrast, *agent-initiated code artifacts*, which remain relatively underexplored, are interactive code objects that agents create, execute, observe, revise, persist, and share within the task execution loop. Through execution feedback, these artifacts help agents reason, act, verify progress, store state, and coordinate with other agents. Examples include regression tests, temporary tools, DSL programs, executable workflows, reusable skills, and intermediate program states. Representative systems such as Claude Code [26], Codex [27], LangChain [28], and enterprise agent platforms show how these elements jointly enable adaptation in long-running agent systems.

With this distinction in mind, we revisit the role of code in agentic systems. Existing surveys typically either treat code as the end product of LLMs. In contrast, we focus on *agent-initiated code artifacts* and how model capabilities construct and evolve them through interaction with harness infrastructure, with code serving as the organizing center for the interface, agent capabilities, and multi-agent coordination. Across diverse agentic systems, code is used not only to produce solutions, but also to execute reasoning, ground actions, maintain state, and expose feedback. We term this view *code as agent harness*: code as the executable and inspectable medium through which agents reason, act, and adapt. This shifts the scope from producing correct programs to understanding how code supports reliable closed-loop agentic behavior.

To systematically characterize *code as agent harness*, we organize the survey into three connected layers, as shown in Figure 1. This organization follows how code becomes an operational medium inside the agent loop: it first enters as a harness interface for reasoning, acting, and environment representation; it then supports harness mechanisms that manage planning, memory, tool use, execution, and repair over time; and it finally becomes a shared artifact through which multiple agents coordinate over repositories, tests, traces, workflows, and execution states.

First, ***Harness Interface: Code for Reasoning, Acting, and Environment Modeling*** (§2) studies how code forms the basic interface between a model and its task environment. At this layer, code is the medium that converts model outputs into executable and inspectable structures. We review *code for reasoning*, where programs externalize intermediate computation and allow interpreters, symbolic solvers, execution traces, or process rewards to check and refine reasoning [7, 6, 8, 29, 30, 31]. We then review *code for acting*, where generated programs serve as policies, tool calls, behavior trees, or reusable skills for embodied, GUI, and software environments [9, 10, 32, 33, 34, 35]. Finally, we examine *code for environment modeling*, where program states, repositories, traces, simulators, and tests represent state, dynamics, and feedback signals for agent interaction [36, 37, 38, 5, 12, 39]. This layer establishes the core harness interface: code is how the agent makes reasoning executable, action programmable, and environment state inspectable.

Building on this interface, ***Harness Mechanisms: Planning, Memory, Tool Use, Control, and Optimization*** (§3) studies how code-harnessed agents remain reliable beyond a single generation step. Once code is placed inside the agent loop, the harness must decide what to execute next, preserve useful state, expose the right tools, and convert failures into corrective actions. We therefore review planning methods that organize long-horizon software tasks through decomposition, structural grounding, trajectory search, or workflow orchestration [40, 41, 42, 43, 44]; memory methods that maintain working state, retrieve repository evidence, store reusable experience, and support shared interaction histories [45, 46, 47, 48]; tool-use methods that connect agents to APIs, repositories, execution environments, and verification tools [19, 49]; and feedback-driven control and harness optimization methods that use static analysis, runtime errors, tests, and human feedback to revise code through repeated execution [50, 51, 52, 53]. This layer turns the interface in §2 into an operational harness: planning controls the execution trajectory, memory preserves state, tools expand the action space, and feedback-driven adaptation closes the loop between failure and revision.

Survey Scope

This survey studies *code as agent harness*: code-centered agent systems where reasoning, action, state, feedback, and verification are organized around executable, inspectable, and stateful programs. We organize the literature up to 2026 into three connected layers:

- **Harness Interface:** code enters the agent loop as a reasoning substrate, an action interface, and an environment representation.
- **Harness Mechanisms:** planning, memory, tool use, control, and harness optimization sustain code-centric agents over long-horizon execution and revision.
- **Scaling the Harness:** shared code artifacts, execution states, repositories, and structured workflows support coordination, review, and collective verification in multi-agent systems.

Finally, *Scaling the Harness: Multi-Agent Orchestration over Code* (§4) extends the harness from a single agent to collaborative ecosystems. When multiple agents operate over code, the harness must not only support individual reasoning and execution, but also coordinate roles, share intermediate artifacts, maintain common state, and verify collective progress. We review multi-agent code-centric systems through agent roles such as manager, planner, coder, reviewer, and tester; collaboration modes such as programming, repair, debate, red-teaming, and adversarial interaction; and workflow topologies ranging from centralized coordination to distributed or streaming collaboration [54, 55, 56]. This layer shows how code becomes a shared harness for orchestrated autonomy: repositories, tests, traces, and structured artifacts provide the common workspace through which agents coordinate, inspect, and improve each other’s behavior.

Beyond the taxonomy, we examine how agent-initiated code interaction appears across five application domains. In coding assistance, agents author patches, tests, and issue-resolution workflows over live repositories [5, 57, 58]. In GUI and OS automation, agents synthesize and execute interface commands grounded in DOM trees, accessibility APIs, and executable evaluators [59, 60]. In scientific discovery, agents dynamically compose and execute hypothesis-testing pipelines spanning simulations, lab protocols, and data analysis [61, 62, 63, 64]. In personalization and embodied control, agents author and revise executable policies, simulators, and skill libraries in response to environment feedback [9, 10, 32]. We further outline open challenges for harness engineering, including evaluation beyond final task success, verification under incomplete feedback, regression-free harness improvement, consistent shared state across multiple agents, human oversight, and extensions to multimodal environments. This survey provides a roadmap for studying code not only as something agents generate, but as the runtime medium through which they execute, adapt, and coordinate reliable behavior.

Contributions

- **Conceptual framing:** We formalize *code as agent harness*, reframing code from a generated artifact into the operational substrate of executable, verifiable, and stateful AI agent systems.
- **Taxonomy and synthesis:** We organize code as agent harness into three connected layers: harness interfaces, harness mechanisms, and scaling harness, and synthesize representative methods.
- **Applications and future agenda:** We connect the taxonomy to real-world applications and outline key challenges in evaluation, verification, safety, and coordination.

Contents

1	Introduction	1
2	Harness Interface: Code for Reasoning, Acting, and Environment Modeling	7
2.1	Code for Reasoning	7
2.1.1	Program-Delegated Reasoning	8
2.1.2	Formal Verification and Symbolic Reasoning Interfaces	9
2.1.3	Iterative Code-Grounded Reasoning	10
2.2	Code for Acting	10
2.2.1	Grounded Skill Selection	12
2.2.2	Programmatic Policy Generation	13
2.2.3	Lifelong Code-Based Agents	13
2.3	Code for Environment	14
2.3.1	Structured World Representations	15
2.3.2	Execution-Trace World Modeling	15
2.3.3	Code-Grounded Evaluation Environments	15
2.3.4	Verifiable Environment Construction	16
3	Harness Mechanisms: Planning, Memory, Tool Use, Control, and Optimization	16
3.1	Planning for Agent Harness	17
3.1.1	Linear Decomposition Planning	18
3.1.2	Structure-grounded Planning	19
3.1.3	Search-based Planning	19
3.1.4	Orchestration-based Planning	20
3.2	Memory and Context Engineering for Agent Harness	21
3.2.1	Working Memory	22
3.2.2	Semantic Memory	23
3.2.3	Experiential Memory	24
3.2.4	Long-Term Memory	24
3.2.5	Multi-Agent Memory	24
3.2.6	Context Compaction and State Offloading	24
3.3	Tool Use for Agent Harness	25
3.3.1	Function-Oriented Tool Use	26
3.3.2	Environment-Interaction Tool Use	27
3.3.3	Verification-Driven Tool Use	27
3.3.4	Workflow-Orchestration Tool Use	28
3.4	Harness Control through the Plan, Execute, and Verify Loop	28
3.4.1	From Debugging to Harness-Level Control	29
3.4.2	Planning as Contract Formation	30
3.4.3	Sandboxed Execution and Permissioned State Transition	30
3.4.4	Verification through Deterministic Sensors	31
3.5	Agentic Harness Engineering for Adaptive Harness Optimization	32
3.5.1	Deep Telemetry as the Optimization Substrate	32
3.5.2	The Evolution Agent	33
3.5.3	Governed Harness Mutation	33

4	Scaling the Harness: Multi-Agent Orchestration over Code	34
4.1	Improved Coding Support through Multi-agent Collaboration	35
4.1.1	Functional Role Specialization and Human-Guided Planning	35
4.1.2	Diverse Interaction Modes Grounded in Shared Program State	38
4.1.3	Optimized Workflow Topology for Agentic Coordination	39
4.2	Execution Feedback and Shared-Harness Synchronization	41
4.2.1	Execution Feedback Integration	41
4.2.2	Shared-Harness Synchronization	42
4.3	Position: The Shared Code-Centric Harness Substrate	43
4.3.1	Shared Harness Representation	44
4.3.2	Harness-State Convergence	45
4.4	Patterns and Trends	47
5	Emerging Fields and Open Problems	49
5.1	Emerging Fields and Tangible Applications	49
5.1.1	Code Assistants	50
5.1.2	GUI/OS Agents as a Program World	53
5.1.3	Autonomous Embodied Agents	56
5.1.4	Agents for Scientific Discovery as Program Worlds	57
5.1.5	Agent Personalization	60
5.2	Open Problems	61
5.2.1	Harness-Level Evaluation and Oracle Adequacy	62
5.2.2	Semantic Verification Beyond Executable Feedback	62
5.2.3	Self-Evolving Harnesses without Regression	63
5.2.4	Transactional Shared Program State and Semantic Conflict Resolution	64
5.2.5	Human-in-the-Loop Safety and Accountability as Harness State	64
5.2.6	Multimodal Code-Harness Systems	65
5.2.7	Toward a Science of Harness Engineering	66

2. Harness Interface: Code for Reasoning, Acting, and Environment Modeling

A harness turns a stateless language model into a functional agent by grounding its outputs in external execution, persistent state, and verifiable feedback. The most fundamental design question for any harness is therefore: *what medium connects the model to its task environment?*

We argue that code is the answer. Unlike natural language, code is *executable*, meaning model outputs become operations with formally verifiable outcomes; *inspectable*, meaning intermediate computation is exposed as structured traces that the harness can read, store, and act upon; and *stateful*, meaning the evolving program represents task progress in a persistent, modifiable form across steps. Crucially, these are not merely properties of code as a notation; they are properties that make code functional as a harness interface. Executability means the harness can verify what the model intended. Inspectability means failures can be diagnosed and fed back. Statefulness means the agent’s interaction history is not lost between steps.

Scope boundary. We use *code* broadly, but not metaphorically. In this survey, code refers to executable or machine-checkable artifacts, including programs, scripts, formal specifications, proof scripts, API schemas, tool definitions, tests, repositories, simulators, configuration files, and code-adjacent execution artifacts such as traces and logs when they are produced by or consumed by executable systems. By contrast, raw perception, physical state, human intent, and model-internal latent reasoning are not themselves code. They may be sensed, estimated, serialized, verified, or acted upon through code, but they should not be conflated with the code interface. This boundary is important because code as a harness interface does not replace perception, embodiment, human goals, or model reasoning; rather, it makes selected aspects of them executable, inspectable, and stateful within the agent loop.

We organize this interface around three roles that code assumes in agentic systems. *Code for reasoning* externalizes internal logic into verifiable computation, allowing external interpreters, symbolic solvers, execution traces, or process rewards to check and refine reasoning (§2.1). *Code for acting* translates high-level intent into executable operations grounded in embodied, GUI, software, or tool-use environments (§2.2). *Code for environment modeling* represents world state, transition dynamics, and feedback signals through program states, repositories, simulators, tests, and logs that agents can execute, edit, and query (§2.3). Overall, these roles define the harness interface: code makes reasoning executable, action programmable, and environment state inspectable.

2.1. Code for Reasoning

A central role of the agent harness is to transform model reasoning from transient text generation into executable and verifiable computation. Early prompting techniques such as pure chain-of-thought (CoT) [65] perform reasoning and computation entirely in *natural language*, forcing the model to both decompose problems and execute intermediate operations within a single latent textual process. While language models are often effective at proposing reasoning steps, they remain unreliable at faithfully carrying out symbolic, logical, or arithmetic computation [7]. More importantly, purely textual reasoning provides the agent harness with little ability to verify intermediate states, inspect execution behavior, or persist computational progress across steps.

Code-for-reasoning thus introduces code as the execution interface between the model and the harness, moving beyond purely text-based reasoning. The model generates executable programs that external runtimes, interpreters, symbolic solvers, or verification modules can execute and evaluate. This separates high-level reasoning from low-level computation: the model proposes procedures, while the harness executes them,

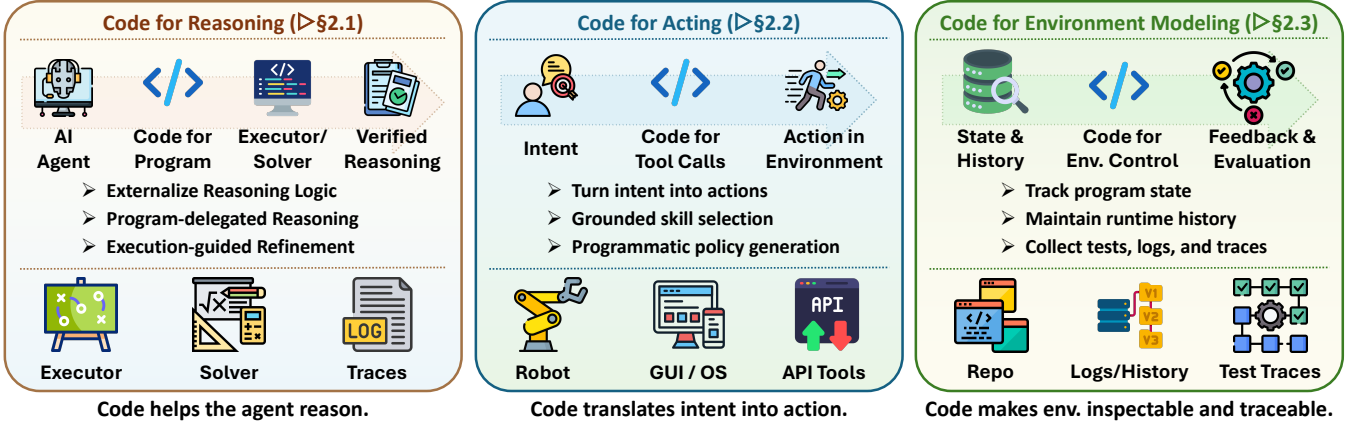


Figure 2: Overview of code as the harness interface, connecting agents to reasoning, action, and environment modeling through executable programs, tool calls, state tracking, and feedback traces.

observes runtime behavior, stores intermediate states, and feeds execution results into future reasoning.

Recent work further broadens this interface from program execution as an external calculator to execution artifacts as reusable reasoning signals. Inputs and outputs, execution traces, variable states, control-flow structures, and function-level tests can all serve as intermediate states that the harness verifies, scores, and feeds back into subsequent reasoning. Existing work can therefore be organized into three paradigms: program-delegated reasoning, formal verification and symbolic reasoning, and iterative code-grounded reasoning. We detail each of them in the following subsections.

2.1.1. Program-Delegated Reasoning

Program-delegated reasoning uses executable programs as the primary interface between problem decomposition and computation. Instead of relying solely on natural language reasoning, the model generates code that external interpreters execute to produce formally grounded outputs. Early works [66, 7] demonstrate that delegating computation to programs substantially improves reliability by moving intermediate reasoning into structured, verifiable execution traces. Program-of-Thoughts (PoT) prompting [6] further systematizes this paradigm by explicitly decomposing reasoning into executable programs, followed by extensions such as POET [67] and MathCoder [68], which improve execution fidelity and domain specialization. Subsequent work investigates the conditions under which program delegation is effective, including the role of execution correctness, task structure, and runtime interaction. For example, Chain of Code (CoC) [8] and CIRS [69] analyze how executable reasoning changes failure modes relative to pure language-based reasoning. Later directions extend this interface beyond isolated task execution. Cross-lingual reasoning frameworks [70] demonstrate that program-based reasoning can generalize across linguistic environments through shared executable structure, while method-based reasoning [71] introduces reusable programmatic procedures that persist across tasks. More recent systems such as CodeAdapt [72] further suggest that tightly coupling language models with executable reasoning interfaces can surpass specialized reasoning-oriented models. Additionally, CodeI/O [73] transforms contextually grounded programs into code input-output prediction tasks, exposing reasoning primitives such as logic-flow planning, state-space search, decision-tree traversal, and modular decomposition while preserving procedural rigor through executable verification.

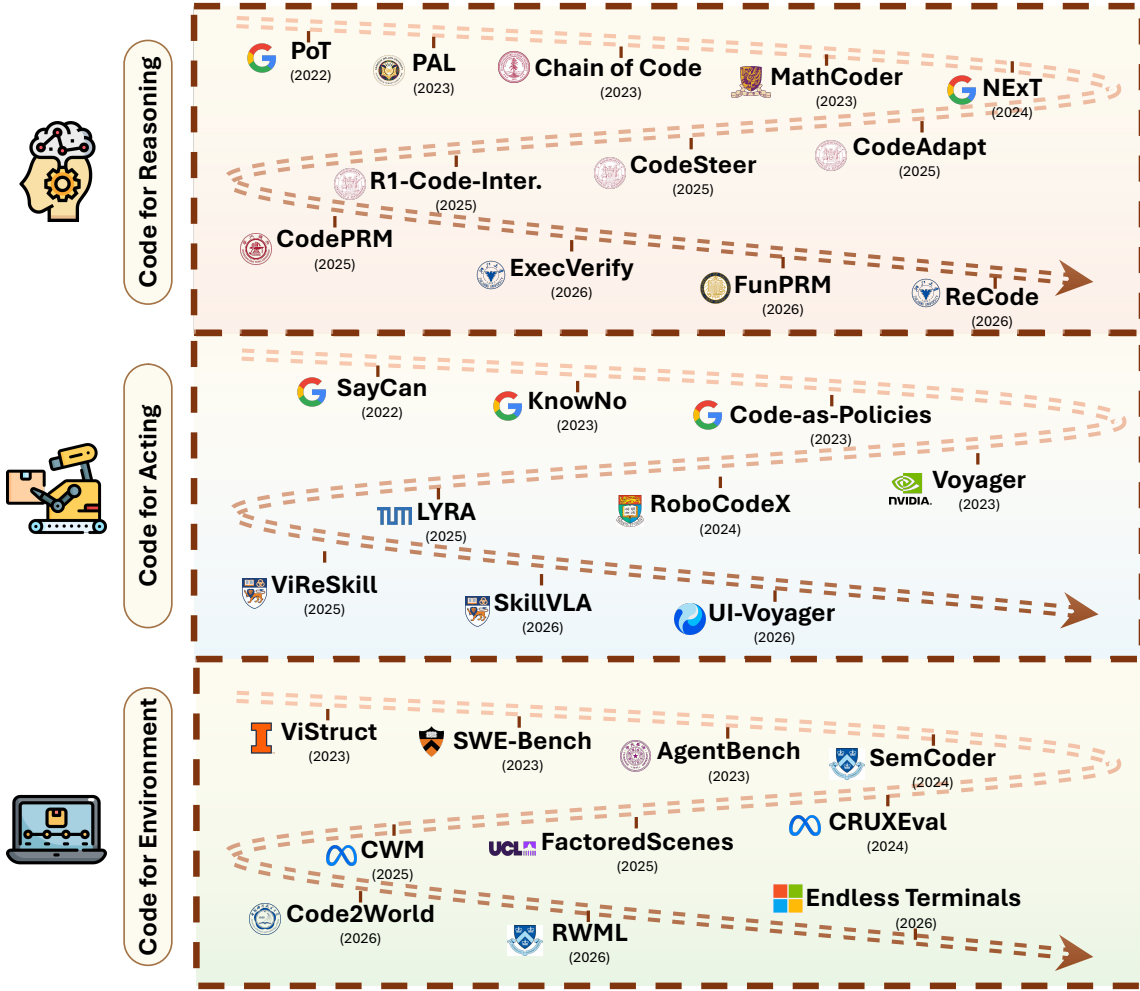


Figure 3: Roadmap of the harness interface, organized by code’s role in reasoning, acting, and environment modeling, with representative works ordered chronologically within each role.

2.1.2. Formal Verification and Symbolic Reasoning Interfaces

Hybrid neural-symbolic methods combine flexible language-based inference with structured symbolic computation, using code and symbolic artifacts as persistent intermediate representations rather than treating programs as mere generated text. Early formulations such as Graph-of-Thoughts [74] generalize chain-of-thought reasoning into graph-structured trajectories, enabling intermediate states to branch, merge, and be reused. Building on this direction, self-verifying reflection [75], MA-LoT [76], and Socratic self-refine [77] introduce iterative verification loops in which symbolic consistency checks guide the refinement of generated solution paths.

Recent work further tightens the coupling between neural generation and symbolic execution through code-based interfaces. CodeSteer [78] and Code-as-Symbolic-Planner [79] explicitly coordinate free-form language reasoning with executable symbolic operations, treating programs as structured substrates that the harness can inspect, transform, and execute across multiple stages. VisualCoder [80] extends this idea by making program behavior visible through control-flow representations. By aligning generated reasoning with visual control-flow graphs and execution paths, it turns dynamic program behavior into an inspectable

artifact for program-behavior prediction. Together, these methods broaden the neural-symbolic interface from textual code to multimodal execution artifacts that a harness can reference, validate, and reuse.

A complementary line of work uses machine-verifiable formal languages as the reasoning interface itself. Proof assistants such as Lean [81], Isabelle [82], and Coq [83] provide formal proof languages based on rigorous logical foundations, enabling each derivation step to be checked by a verifier. Early LLM-based theorem-proving systems, including ReProver [84], DeepSeek-Prover [85], and TheoremLlama [86], establish practical recipes for combining language models with proof-assistant feedback in mathematical reasoning. More recent systems, such as DeepSeek-Prover-V2 [87], Kimina-Prover [88], MA-LoT [76], and Goedel-Prover-V2 [89], improve this process through deliberative proof search, self-correction, and repeated proof generation and verification. Formal verification interfaces are also expanding beyond theorem proving in mathematics. HybridReasoning [90] applies formal provers to support natural-language reasoning; Lean4Physics [91] and PhysLib [92] extend Lean-based verification to physics; and VERINA [93] and Goedel-Code-Prover [94] adapt formal methods to code verification. Lean4Agent [95] further extends this trajectory to agentic systems by using Lean4 to model and verify agent workflows and trajectories. From the harness perspective, these systems show how formal languages can serve not only as reasoning tools, but also as executable contracts that constrain, certify, and audit agent behavior.

2.1.3. Iterative Code-Grounded Reasoning

Iterative code-grounded reasoning focuses on closed-loop interaction between generation, execution, and feedback. In these systems, reasoning is not a single-pass process, but an iterative computational trajectory grounded in executable state transitions. Early work such as NExT [30] trains models to anticipate execution behavior by reasoning over program traces, thereby grounding intermediate reasoning in runtime semantics. Related efforts [96] similarly emphasize that executable traces provide a richer supervision signal than final textual outputs alone. Building on this foundation, subsequent approaches introduce explicit generate–execute–verify–refine loops. Methods such as CodePRM [31] and ORPS [97] use execution outcomes to evaluate and refine intermediate reasoning trajectories, enabling the harness to guide reasoning through runtime feedback rather than pure next-token prediction. Along the same direction, systems such as CYCLE [98] and Self-Edit [99] iteratively revise generated solutions using execution-aware correction signals. Reinforcement learning further strengthens this paradigm by treating execution feedback as an optimization signal over reasoning trajectories. Methods such as CodeRL [100], CodeRL+ [101], and RLTF [102] optimize functional correctness through unit-test-based rewards, while approaches such as StepCoder [103] incorporate fine-grained compiler and runtime feedback during optimization. RLEF [104] formalizes this interaction as policy optimization grounded in multi-step execution feedback, allowing reasoning policies to adapt through iterative runtime interaction. More recent approaches move toward fully interactive reasoning environments. For example, EG-CFG [21] injects execution signals directly during generation to support step-level correction, while systems such as R1-Code-Interpreter [105] interleave reasoning and multiple rounds of code execution within persistent interactive sessions.

2.2. Code for Acting

Beyond reasoning, the agent must also connect the model to external environments where decisions produce real executable effects. At this stage, code no longer serves primarily as a medium for computation, but as an action interface that converts model outputs into grounded operations such as tool invocations, robot-control policies, GUI actions, or software commands. Through this interface, the harness translates high-level intent into executable behaviors that can interact with embodied, digital, and interactive environments. The

Table 1: Representative systems where code serves as a reasoning substrate.

Method	Mechanism	Reasoning Paradigm	Key Innovation
PoT [6]	Delegated	Hybrid comments	Merges code with natural language CoT
PAL [7]	Delegated	Program-aided	Decouples logic from computation
CodeAdapt [72]	Delegated	Generalizable logic	Code-enabled LLMs outperforming reasoning models
CodeI/O [73]	Delegated	I/O prediction	Converts code into verifiable input-output reasoning tasks
SATLM [29]	Formal	SAT/SMT solving	Uses symbolic solvers as machine-checkable reasoning backends
ReProver [84]	Formal	Lean proof search	Combines LLM generation with proof-assistant feedback
Dpsk-Prover [85]	Formal	Lean theorem proving	Trains LLMs for formal mathematical proof generation
Dpsk-Prover-V2 [87]	Formal	Deliberative proving	Lean proof search through decomposition and self-correction
Goedel-Code-Prover [94]	Formal	Lean code proof	Searches hierarchical Lean proofs for code verification
Lean4Agent [95]	Formal	Agent verification	Models and verifies agent workflows and trajectories in Lean4
Chain of Code [8]	Hybrid	LMulator	Simulates non-executable semantic code
SATLM [29]	Hybrid	Formal Logic	Uses SAT/SMT solvers as reasoning backend
CodeSteer [78]	Hybrid	Symbolic control	Explicitly transitions between symbolic code and neural text
VisualCoder [80]	Hybrid	CFG-grounded	Aligns code reasoning with visual control-flow artifacts.
NExT [30]	Iterative	Trace-grounded	Anticipates execution behavior via program traces
MathCoder [68]	Iterative	Feedback-driven SFT	Interleaves code, output, and reflection
CodePRM [31]	Iterative	Process rewards	Learns reward functions over reasoning-execution trajectories
RLEF [104]	Iterative	Multi-step RL	Optimizes policy directly using execution feedback
EG-CFG [21]	Iterative	Execution-guided	Integrates execution signals directly during generation
R1-Code-Int. [105]	Iterative	Fully interactive	Autonomously interleaves reasoning and multiple executions
ExecVerify [106]	Iterative	Stepwise RL	Uses statement- and variable-level execution rewards.
FunPRM [107]	Iterative	Function-step PRM	Treats functions as verifiable process-reward units.
ReCode [108]	Iterative	Process RL	Reinforces code generation with reasoning-process rewards

central challenge is therefore grounding: the harness must map abstract language outputs into executable behaviors that respect the constraints of the target environment, including embodiment limits, interface APIs, environment dynamics, and safety requirements. Unlike code-for-reasoning, where interpreters can often directly verify correctness, action execution occurs in partially observed and dynamically evolving environments, where failures may emerge through invalid state transitions, delayed feedback, or silent execution errors. For example, a robot may attempt to grasp an object outside its reachable workspace without producing an explicit runtime exception.

Importantly, executable action code is an interface to these components, not a replacement for them. In embodied settings, perception modules provide observations, affordance or feasibility models estimate which actions are possible, motion planners and controllers connect symbolic commands to sensors and actuators, and safety layers constrain dangerous or invalid behavior. In GUI and software settings, the analogous components include screen parsers, DOM or accessibility trees, backend APIs, user-intent models, permission systems, and programmatic validators. Code sits between the model and these components: it serializes observations, calls grounding and planning modules, invokes executable actions, and exposes validation results back to the harness.

Code-for-acting therefore introduces structured executable programs as the control interface between the model and the environment, allowing the harness to execute, monitor, validate, reuse, and refine actions through interaction feedback. This interface can be realized in different forms: a predefined skill library, a

Table 2: Representative systems where code serves as an action interface.

Method	Mechanism	Action Paradigm	Key Innovation
AutoHarness [109]	Harness Gen.	Action validation	Synthesizes code harnesses that mediate model actions and filter invalid environment interactions
SayCan [9]	Skill Selec.	Affordance-based	Links LLM plans to physical feasibility
KnowNo [110]	Skill Selec.	Conformal prediction	Calibrates planner uncertainty for ambiguous instructions
SkillVLA [111]	Skill Selec.	Bimanual grounding	Extends grounding to combinatorial skill reuse
BOSS [112]	Skill Selec.	Skill bootstrapping	Synthesizes new executable skill chains via guided practice
LLM-Guided Traj. [113]	Skill Selec.	Trajectory generation	Generates diverse manipulation trajectories and executable success conditions
LRLL [114]	Skill Selec.	Lifelong grounding	Evolving skill interface via memory and self-exploration
CaP [10]	Policy Gen.	Hierarchical Python	Generates reactive robot control policies
RoboCodeX [33]	Policy Gen.	Multimodal tree	Synthesizes tree-structured code across navigation
Code-BT [34]	Policy Gen.	Behavior-tree	Imposes rule constraints via code-to-behavior-tree planning
ALRM [115]	Policy Gen.	Closed-loop control	Integrates programmatic generation with ReAct execution
CP-Agent [116]	Policy Gen.	Constraint solving	Uses persistent execution loops for formal constraint-model repair
Robot-Code Sim. [117]	Policy Gen.	Static simulation	Uses LLMs as static simulators for robot code evaluation
GenSwarm [118]	Policy Gen.	Multi-robot control	Coordinates policy generation and deployment across robotic agents
NormCode [119]	Policy Gen.	Governed interface	Enforces auditability and data isolation through semi-formal code
RACAS [120]	Policy Gen.	Cooperative control	Robot-agnostic architecture for closed-loop cooperative agents
Voyager [32]	Lifelong	Skill Library	Autonomous curriculum for open-ended tasks
LYRA [121]	Lifelong	Human-in-loop	Encodes human corrections into reusable structured skills
ViReSkill [122]	Lifelong	Vision-grounded	Replanning on failure using a skill-memory cache
UI-Voyager [35]	Lifelong	Self-evolving	Rejection fine-tuning and self-distillation for mobile GUI agents
SkillsCrafter [123]	Lifelong	Continual skills	Mitigates forgetting as executable manipulation skills accumulate

generated control policy, a persistent skill memory, a GUI/API tool protocol, or an explicit action-validation harness. AutoHarness [109] makes the last form explicit by automatically synthesizing a code harness that mediates between the LLM and the environment, filtering invalid actions before execution. This highlights the core harness view of code-for-acting: code is not only the action to be executed, but also the executable boundary that connects model intent to perception, grounding, affordance estimates, controllers, APIs, actuators, and safety constraints.

2.2.1. Grounded Skill Selection

Grounded skill selection studies how the agent maps high-level language intent into executable behaviors through reusable skill interfaces. Rather than generating low-level actions directly, these systems treat the environment as a collection of executable capabilities that the agent harness can invoke, compose, and refine under environmental constraints. SayCan [9] establishes the core paradigm by coupling language planning with grounded skill execution, allowing the agent to select actions based not only on semantic relevance but also embodiment feasibility. Subsequent work extends this execution interface in several directions. KnowNo [110] introduces uncertainty-aware control through conformal prediction, enabling the harness to detect ambiguous states and trigger clarification before unsafe execution. BOSS [112] addresses the rigidity of fixed skill libraries by using language-guided practice to synthesize new executable skill chains, allowing the harness to expand its action space over time. Similarly, [113] tackles the data bottleneck of grounded interaction by using LLM-guided generation to construct diverse manipulation trajectories

and executable success conditions for automatic retry and relabeling. Beyond static execution, LRL [114] introduces memory and self-guided exploration to maintain a persistent and evolving skill interface across tasks. Finally, SkillVLA [111] extends this paradigm to combinatorial bimanual interaction, emphasizing that grounded action interfaces must support structured skill reuse and recomposition under increasingly complex embodiment settings.

2.2.2. Programmatic Policy Generation

Programmatic policy generation treats code itself as the control interface between the model and the environment. Instead of selecting from predefined skills, the harness directly materializes executable policies as programs that specify control logic, perception-conditioned branching, feedback loops, and API interaction. CaP [10] crystallizes this paradigm by framing LLM-generated Python programs as executable robot policies. Building on this idea, RoboCodeX [33] introduces multimodal and tree-structured code generation to support more complex manipulation and navigation behaviors. Subsequent work focuses on scaling the interaction substrate. RoboPro [124] synthesizes executable policy code from large-scale in-the-wild videos, while CodeBT [34] compiles generated programs into behavior-tree controllers that support constrained execution and iterative runtime feedback. Beyond robotics, CP-Agent [116] demonstrates that persistent execution loops can support formal constraint-solving agents through iterative execution and repair. To reduce dependence on expensive physical environments, [117] configures language models as static execution simulators for robot code evaluation. GenSwarm [118] further extends programmatic control to multi-agent robotic systems, where the harness must coordinate policy generation, constraint analysis, and deployment across multiple embodied agents. At the systems level, NormCode [119] emphasizes governance and auditability by introducing a semi-formal programming interface with enforced data isolation, allowing execution traces and control logic to remain inspectable and constrained. Finally, ALRM [115] and RACAS [120] consolidate these ideas into persistent closed-loop control architectures that integrate code generation, execution, monitoring, and iterative interaction within unified agent harnesses.

2.2.3. Lifelong Code-Based Agents

Lifelong code-based agents study how executable interaction interfaces can persist, evolve, and accumulate capabilities over long-horizon interaction. In these systems, code is not only an execution mechanism, but also a persistent memory substrate through which the harness stores reusable behaviors, interaction traces, and environment knowledge. Voyager [32] establishes this paradigm through an automatic curriculum and continually expanding executable skill library for open-ended interaction in Minecraft. Extending this idea to embodied environments, LRL [114] introduces persistent memory, self-guided task exploration, and skill abstraction to overcome the limitations of fixed policy libraries without requiring gradient updates. A central challenge in lifelong harnesses is that interaction feedback and corrections are often transient and difficult to reuse. LYRA [121] addresses this issue by converting human corrections into reusable executable skills and retrieval-augmented memory structures. Similarly, ViReSkill [122] combines vision-grounded replanning with skill-memory caching to maintain stable interaction under environmental failures and output variability. Recent work further focuses on continual adaptation and self-evolution under persistent deployment. SkillsCrafter [123] introduces continual language-conditioned manipulation structures to mitigate catastrophic forgetting as executable capabilities accumulate, while UI-Voyager [35] generalizes the self-evolving interaction paradigm to GUI agents through failure-driven adaptation and self-distillation. Together, these systems move beyond one-shot execution toward persistent agent harnesses that continuously expand, refine, and reuse executable interaction interfaces over time.

Table 3: Representative systems where code serves as an environment representation.

Method	Mechanism	Environment Paradigm	Key Innovation
ViStruct [125]	Structured	Class/object hierarchy	Encodes visual scenes as data structures
FactoredScenes [126]	Structured	Room programs	Composes object/relation functions for 3D layout generation
PoE-World [127]	Structured	Programmatic experts	Scales symbolic world models beyond simple grid-worlds
Code2World [38]	Structured	Render-aware RL	Re-frames GUI state prediction as renderable HTML generation
SemCoder [128]	Trace-based	Semantic alignment	Pairs code with detailed execution traces
WorldCoder [36]	Trace-based	Model-based RL	Synthesizes transition and reward models
CWM [37]	Trace-based	Open-weights trace	Trains large LLMs natively on program execution traces
RWML [129]	Trace-based	Self-supervised RL	Aligns simulated next states with realized environment states
AWM [130]	Trace-based	World-modeling	Aligns multiple executable world models across tasks
WorldMind [131]	Trace-based	Model fusion	Coordinates executable world models from knowledge sources
SWE-bench [5]	Evaluation	Repo-level testing	Uses unit tests as objective world states
AgentBench [12]	Evaluation	Multi-env interaction	Benchmarks across OS, databases, and games
CRUXEval [132]	Evaluation	Execution tasks	Benchmarks functional input and output prediction
End Terms. [39]	Evaluation	Procedural RL envs	Automates generation of terminal-use evaluation tasks
InterCode [11]	Evaluation	Interactive execution	Frames coding tasks as actions with sandbox feedback
LiveCodeBench [133]	Evaluation	Live coding eval	Continuously updates execution-based evaluation pipelines
CRUXEval-X [134]	Evaluation	Multilingual execution	Extends input-output execution evaluation across languages
CoRe [135]	Evaluation	Runtime reasoning	Evaluates code reasoning through execution-centered tasks
CodeGlance [136]	Evaluation	Multimodal code eval	Evaluates code understanding under visual and structural settings
SWE-smith [137]	Construction	Synthetic SWE envs	Generates repository-level tasks and execution environments
EnvScaler [138]	Construction	Tool-interactive envs	Synthesizes tool-use environments with programmatic validators

2.3. Code for Environment

The agent must also maintain an explicit representation of the environment with which the agent interacts. Without such a representation, the environment is exposed to the agent only indirectly through textual observations, API returns, or sparse feedback signals. As a result, environment state often remains implicit, transient, and difficult to verify, making it challenging to track state transitions, evaluate interaction outcomes, or reuse past interaction history across long-horizon tasks. This limitation becomes particularly severe in complex software, robotic, and multi-step interactive environments, where successful interaction depends on maintaining consistent world state and grounded feedback over time.

Code-for-environment addresses this limitation by introducing executable programs as the environment interface itself. Instead of treating the environment as an opaque external process, these systems materialize environment structure and dynamics through computational artifacts such as simulators, repositories, tests, execution traces, logs, and state-transition programs. This allows the agent to explicitly store, inspect, execute, and modify environment state throughout interaction. Representing environments through executable code provides two major advantages. First, executable environments expose verifiable state transitions, allowing the agent to evaluate interaction outcomes through execution rather than ambiguous natural-language judgment. Second, code-based environments are persistent and modifiable that agents can query, simulate, edit, and refine during interaction. Rather than interacting with an opaque world solely through language, agent harness can ground reasoning and action in explicit computational state and runtime dynamics. Existing work in this direction can be organized into four paradigms: structured world representations, execution-trace world modeling, code-grounded evaluation environments, and verifiable environment construction.

2.3.1. Structured World Representations

Structured world representations model environments through explicit programmatic structures that the agent can execute, inspect, and manipulate. Rather than representing the environment solely through latent embeddings or textual descriptions, these approaches encode world state, object relations, spatial layouts, and interaction dynamics as structured computational artifacts. For example, ViStruct [125] uses programming-language structure as an explicit interface for visual structural knowledge extraction, enabling multi-granular visual events to be represented through consistent executable structures. FactoredScenes [126] similarly models indoor environments as compositional “room programs,” where reusable object and relation functions define physically consistent scene layouts. Extending this idea to scalable symbolic world modeling, PoE-World [127] introduces a compositional framework that combines many small programmatic experts to represent increasingly complex environment dynamics. More recent systems broaden structured environment interfaces to high-fidelity interactive worlds. Code2World [38] reframes GUI state prediction as renderable HTML generation, allowing environment transitions to be represented and evaluated through executable rendering code. Code2Worlds [139] further extends this paradigm to 4D simulated environments through language-to-simulation program generation, where physics-aware execution loops reduce semantic-physical inconsistencies during environment construction and interaction.

2.3.2. Execution-Trace World Modeling

Execution-trace world modeling studies how the agent can learn environment dynamics directly from executable interaction traces. Instead of treating execution merely as a final evaluation step, these approaches model runtime transitions themselves as the primary representation of environment behavior. SemCoder [128] bridges static programs and runtime semantics by training language models to reason about functional behavior, statement-level execution effects, and input-output transitions. Building on this perspective, Code World Model (CWM) [37] learns predictive world models directly from program traces, enabling the agent to anticipate future environment states through executable dynamics. WorldCoder [36] further introduces a model-based interaction framework in which the agent explicitly writes and updates executable world models represented as Python programs. Rather than storing environment knowledge implicitly in model parameters alone, the agent maintains editable computational representations that can be executed, revised, and reused during planning and interaction. Subsequent work extends this paradigm toward continual and interactive world-model adaptation. RWML [129] combines execution traces with reinforcement learning to refine environment dynamics through runtime interaction, while AWM [130] and WorldMind [131] study how multiple executable world models can be aligned, fused, and coordinated across tasks and knowledge sources.

2.3.3. Code-Grounded Evaluation Environments

Code-grounded evaluation environments use executable systems as the interface for measuring agent behavior and interaction quality. Unlike static benchmarks based solely on textual outputs, these environments expose explicit runtime state transitions, execution feedback, and verifiable interaction outcomes that the agent can directly observe and evaluate. InterCode [11] establishes this paradigm by reframing coding tasks as interactive execution environments, where code acts as actions, execution feedback serves as observations, and sandboxed runtimes provide grounded interaction. CRUXEval [132] further evaluates program understanding through executable input-output prediction tasks, while LiveCodeBench [133] introduces continuously updated evaluation pipelines that assess execution, self-repair, and runtime reasoning capabilities under evolving problem distributions. SWE-bench [5] extends executable evaluation to real-

world software repositories, where agents must modify large-scale codebases and are evaluated through repository-level unit-test execution rather than textual correctness alone. More broadly, AgentBench [12] demonstrates that executable interaction environments can evaluate reasoning and decision-making across diverse embodied and digital tasks. Subsequent benchmarks such as CRUXEval-X [134], CoRe [135], GeoGramBench [140], CodeGlance [136], and Endless Terminals [39] further expand this paradigm toward multilingual, multimodal, and continuously interactive evaluation settings, where runtime interaction rather than static answer matching becomes the primary evaluation interface.

2.3.4. Verifiable Environment Construction

A newer direction treats executable environments not only as benchmarks to evaluate agents, but as harness artifacts that can be synthesized, scaled, and validated programmatically. This is especially important for long-horizon agents, where the harness must provide not only a task prompt, but also a runnable state, transition dynamics, feedback channels, and verification oracles. SWE-smith [137] scales software-engineering agent data by constructing repository-level tasks and execution environments from existing codebases, turning software repositories into reproducible program worlds for agent training and evaluation. EnvScaler [138] extends this idea beyond software engineering by programmatically synthesizing tool-interactive environments together with scenarios and rule-based trajectory validators. From the harness perspective, these methods make the environment interface itself an object of construction: code specifies not only what the agent edits or executes, but also the state transitions, tool affordances, and verifiers that determine whether an interaction has succeeded.

3. Harness Mechanisms: Planning, Memory, Tool Use, Control, and Optimization

Harness mechanisms form the central systems layer that makes code-harnessed agents reliable beyond a single generation step. Once code enters the agent loop, software generation is no longer only a problem of producing correct programs from a prompt. It becomes an interaction among the model, mutable task state, and human-designed harness infrastructure. The model provides judgment: it decomposes goals, selects actions, interprets feedback, and decides when to revise. Mutable state records repository evidence, working context, execution traces, validation results, memories, and intermediate beliefs about the task. The harness infrastructure exposes tools and execution substrates, persists and compacts state, constrains actions through policies and permission tiers, routes feedback, and verifies whether each state transition is acceptable. From this perspective, harness mechanisms are not isolated add-on modules, but coordinated control surfaces that turn model decisions into bounded, observable, and revisable changes in an executable environment. In its basic form, code allows the agent to call existing executable interfaces. Further, the agent can dynamically author task-specific executable interfaces. These agent-authored artifacts make the harness more adaptive because they allow the execution environment to be reshaped around the current task. However, dynamically authored code does not replace the broader human-designed harness infrastructure. Reliability still depends on model-side judgment together with human-designed policies, sandbox boundaries, permission tiers, verification oracles, audit logs, and human-review gates. Code therefore serves as an executable medium inside the harness, while the harness remains the larger policy-governed system that decides what code may be executed, trusted, persisted, reused, or promoted into future workflows.

In this section, we review five interacting categories of harness mechanisms for code agents. Planning (§ 3.1) organizes long-horizon task execution by externalizing goals into decompositions, structural constraints, search trajectories, or workflow-level orchestration. Memory and context engineering (§ 3.2) manage mutable state across long interactions by preserving working context, retrieving repository evidence, storing

Code as Agent Harness

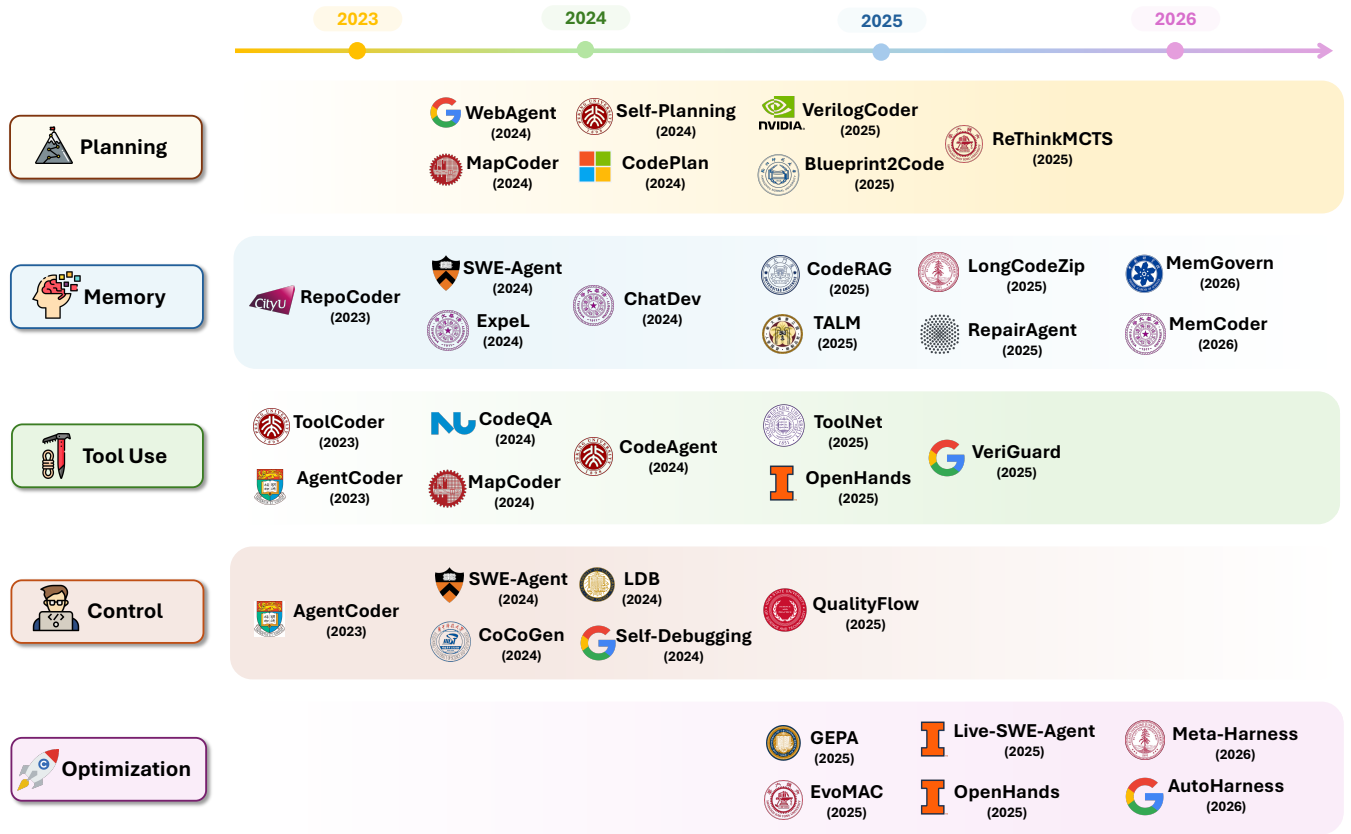


Figure 4: A roadmap overview of agent harness mechanisms.

reusable experience, supporting shared histories, and offloading state beyond the active context window. Tool usage (§ 3.3) connects the agent to governed executable interfaces, including APIs, repositories, terminals, sandboxes, verification tools, and workflow orchestrators. Harness control through the Plan-Execute-Verify loop (§ 3.4) reframes feedback-guided debugging as a broader control process: plans form contracts over intended changes, execution applies them inside sandboxed and permissioned environments, and verification uses deterministic sensors and human-review gates to decide whether the state should be accepted, revised, escalated, or rolled back. Finally, agentic harness engineering (§ 3.5) studies how the harness itself can be measured and improved through deep telemetry, evolution agents, replay-based evaluation, and governed harness mutation.

3.1. Planning for Agent Harness

Planning plays a central role in agentic harness because real-world software engineering tasks rarely admit a direct one-shot mapping from natural language intent to correct implementation. From the harness perspective, planning is not merely an internal reasoning capability of the LLM, but a form of *harness control*: it structures how the agent externalizes intent into executable steps, schedules interactions with code artifacts and tools, and regulates the trajectory of reasoning, execution, and revision over time. Beyond generating code tokens, an effective agent harness must organize long-horizon problem solving into a coherent course of action, deciding what intermediate goals to pursue, in what order to execute them, what artifacts to inspect or modify, and how to revise the trajectory when execution feedback reveals errors, missing dependencies, or

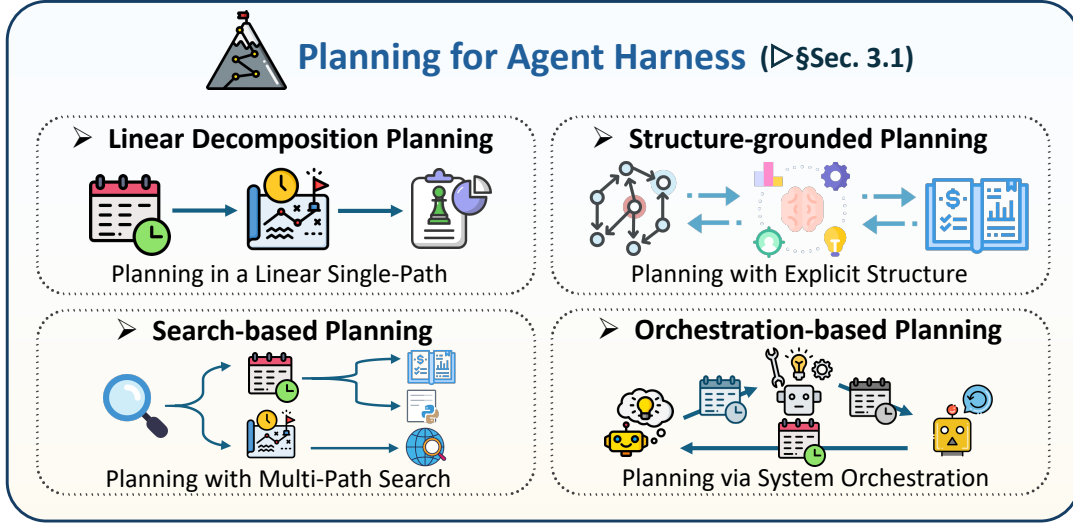


Figure 5: Overview of planning mechanisms for agent harnesses.

violated constraints. This need becomes especially pronounced in repository-level editing, web interaction, competitive programming, and hardware design, where the agent must operate over large action spaces, sparse feedback, and deeply interdependent subproblems. In such settings, a fundamental challenge arises **between the complexity of the target task and the limited reliability of unconstrained agent execution**: without an explicit planning mechanism as harness control, the agent may commit too early to brittle solution paths, overlook latent dependencies, or fail to coordinate reasoning, retrieval, execution, and revision into a stable workflow.

Early planning-oriented systems mainly treated planning as a linear decomposition step, where the model first produced a natural-language solution outline and then translated it into code. As code agents were applied to more complex environments, however, planning gradually evolved from a simple pre-generation scaffold into a richer harness-level control mechanism. It can be grounded in repository structure or external knowledge to constrain the agent’s action space, expanded through explicit search over multiple candidate trajectories to improve robustness, or distributed across specialized agent roles and feedback loops to coordinate execution at the system level. Based on the **primary locus where harness control is realized**, we categorize existing planning methods in code agents into four types: *linear decomposition planning*, *structure-grounded planning*, *search-based planning*, and *orchestration-based planning*.

3.1.1. Linear Decomposition Planning

In this planning paradigm, the agent first produces a single explicit, executable sequence of steps, and then carries out generation by following this decomposition [141, 40, 41, 142, 143]. A lightweight precursor of this pattern is ReAct [144], where the agent interleaves thoughts, actions, and observations in a serial trajectory. In this framework, each reasoning step externalizes the current subgoal and constrains the next action, turning the trajectory itself into a stepwise harness for control. This pattern is most directly instantiated in Self-Planning [40]: the model first decomposes the intent into concise, high-level numbered steps, and then generates code step by step under the guidance of this plan. Plan-And-Act [145] further makes this harness explicit by separating a planner, which produces structured high-level plans: the planner repeatedly refreshes the linear scaffold as new observations arrive, allowing the planning strategy to preserve task-level control while adapting to environmental feedback. WebAgent [41] extends this idea to web

automation: it decomposes a user instruction into successive sub-instructions, summarizes task-relevant HTML conditioned on the current subgoal, and then synthesizes executable Python actions from that linear sub-instruction sequence. KareCoder [141] follows a similar template in a knowledge-augmented setting, where the model first constructs a knowledge-aware, step-by-step prompt from an external knowledge library and then uses this prompt to generate code, making planning a structured intermediate layer between problem understanding and implementation. Recent industrial practice shows that this linear scaffold can be lifted from an ephemeral prompt artifact to a persistent harness object. In long-horizon coding workflows, files such as `PLAN.md`, `Implement.md`, and status logs record milestones, acceptance criteria, validation commands, and recovery rules, allowing the agent to reload, update, verify, and document progress across context resets or multi-session execution [146, 147]. In this view, planning is no longer merely an internal reasoning trace, but a filesystem-backed control object: it can be reviewed by humans, versioned with Git, consumed by subagents, and used as the source of truth for implementation. The main limitation remains that these methods typically commit to a single decomposition trajectory: when the initial plan is incomplete or misaligned, the harness can improve persistence and auditability, but it still provides limited exploration beyond the chosen path.

3.1.2. Structure-grounded Planning

In this line of work, the agent does not derive its action sequence solely from a free-form natural language prompt, but instead grounds planning in an explicit structured representation of the task environment, such as dependency graphs, repository graphs, circuit graphs, or knowledge graphs. These structures act as natural harness scaffolds: they expose relevant entities, encode dependency relations, and guide the order in which subtasks should be generated, revised, or verified. For example, CodePlan [42] constructs a plan graph over edit obligations and derives new steps through dependency analysis and change-impact propagation. Meanwhile, repository understanding methods [148, 149, 150, 148] convert codebases into heterogeneous graphs or text-rich code graphs, then use graph-integrated reasoning to localize relevant entities and condition downstream generation on structural dependencies rather than flat text context. GraphCodeAgent [151] extends this idea with a dual-graph harness, where a Requirement Graph captures relations among natural-language requirements and a Structural-Semantic Code Graph captures repository dependencies. The same principle also appears in recent agent-native repository practices. Files such as architecture notes, API specifications, and testing guides turn project knowledge into persistent, inspectable, and version-controlled artifacts that the agent can consult before acting [152, 153, 154]. This broadens structure-grounded planning beyond graph construction: the relevant structure determines explicit rules, build commands, directory boundaries, coding conventions, and design constraints, thereby promoting a coherent and stable harness control over the programs. Specialized domains follow the same pattern [155, 156]. VerilogCoder [156] grounds subtask planning in a Task and Circuit Relation Graph so that each subtask is enriched with signals, transitions, and examples, while DomAgent [155] uses knowledge graphs to combine top-down structured knowledge with bottom-up examples for domain-specific code generation. Overall, these works show that structure-grounded planning improves coherence, dependency awareness, and long-horizon consistency by turning project or domain knowledge into explicit and inspectable harness objects that guide the agent’s behavior over time.

3.1.3. Search-based Planning

Search-Based Planning allocates inference-time compute to systematically explore, evaluate, and select among multiple candidate solution paths. Rather than committing the agent to a single plan, the key idea is to expand the decision space and use feedback to control which alternatives should be pursued, revised, or

Table 4: Representative planning modules for code agents.

Method	Category	Core Mechanism	Interface	Feedback
Self-Planning [40]	Linear decomposition	Stepwise decomposition	Shared prompt	None
WebAgent [41]	Linear decomposition	Sub-instruction sequencing	APIs	Runtime exception
CodePlan [42]	Structure-grounded	Plan graph	Repo graph	Critique
VerilogCoder [156]	Structure-grounded	Task-circuit relation graph	Repo graph	Test pass/fail
Tree-of-Code [159]	Search-based	Trajectory tree search	Execution env	Test pass/fail
ReThinkMCTS [158]	Search-based	MCTS over reasoning paths	Execution env	Critique, tests
MapCoder [44]	Orchestration-based	Role orchestration	APIs	Critique, tests
Blueprint2Code [165]	Orchestration-based	Blueprint-to-code	Repo interface	Critique

discarded. A first group of methods [157, 158] instantiates this harness in the thought space. Instead of directly writing code, they first branch over high-level observations, strategies, or reasoning traces, with the goal of increasing conceptual diversity before implementation. In this view, better planning comes from covering a broader idea space and using feedback to refine reasoning itself, rather than merely repairing final code. A second group [43, 159, 160, 161] performs search in the trajectory space of coding actions: these methods model coding as a branching process over strategy choice, implementation, debugging, and revision, and rely on execution signals or learned critics to decide which nodes to expand. Therefore, long-horizon coding quality improves when the agent can backtrack from suboptimal decisions and compare partial trajectories. Another line of these works, such as ReLoc [162] and SFS [163], treats planning as search in code space. Here the methods iteratively explore neighboring programs through mutation, revision, or local optimization, guided by validation feedback or fine-grained scoring signals. Beyond the above methods, recent systems increasingly treat candidate plans, patches, logs, tests, and execution traces as persistent artifacts rather than transient generations. SWE-Search [164] combines Monte Carlo Tree Search with software-engineering agents to explore alternative repair trajectories, while CodeTree [43] organizes strategy exploration, solution generation, and refinement within a unified tree. More broadly, Meta-Harness [13] pushes this idea to the harness level itself: it searches over harness code by giving an agent access to prior source code, scores, and execution traces through a filesystem. These developments suggest that search-based planning is not only a model-side sampling strategy, but also a harness-level state management problem: the runtime must preserve candidates, expose evidence, run validators, and decide which branch deserves further computation.

3.1.4. Orchestration-based Planning

Orchestration-Based Planning refers to a planning paradigm in which the core planning function is realized through a harness design for system-level coordination. In this paradigm, the harness governs how agents or modules specialize roles, execute stages, route feedback, and trigger verification loops, thereby determining what actions should be taken next in long-horizon code generation workflows. A first common pattern [50, 51, 52] is feedback-centered orchestration, where the system distributes coding, testing, analysis, and repair across different modules, so that progress is driven by repeated execution-grounded feedback and adaptive escalation. In this group, planning is not an up-front artifact, but an emergent property of how failures are detected, interpreted, and routed back into subsequent actions. A second pattern [44, 166, 165] is staged workflow orchestration, which casts code generation as a structured software-process pipeline, such as comprehension, retrieval or preview, planning or blueprinting, coding, debugging, and repair. The main advantage of this group lies in decomposing complex generation into interpretable stages with explicit handoff rules, and the actual planning power comes from cross-stage control, candidate pruning, and

iterative refinement. A third pattern [167, 168, 169, 170] is controller-centric orchestration, where planning is embedded in the transformation of intermediate artifacts and in the routing substrate itself. Here, systems organize decision-making through mechanisms such as formal-specification pipelines, suggestion stages between localization and repair, typed intermediate representations, shared blackboards, or specialized planner–coder coordination, so that the next plan is determined by the scaffold’s control logic rather than by a single textual prompt.

Recent harness systems make this orchestration view especially explicit. Anthropic’s long-running harnesses separate planning, generation, and evaluation into distinct roles, using structured artifacts and independent evaluation to maintain progress across long sessions [15, 171]. Cursor’s large-scale autonomous coding experiments similarly highlight planner–worker coordination as a way to scale from focused single-agent tasks to many parallel agents working on a shared project [172]. The most general formulation appears in Natural-Language Agent Harnesses, where high-level harness logic (such as roles, stages, contracts, adapters, state conventions, and failure taxonomies) is written as editable natural language and executed by an Intelligent Harness Runtime [173]. The IHR interprets these high-level natural-language instructions at runtime and converts them into constrained execution steps under explicit contracts, budgets, tool interfaces, and environment state. This reframes orchestration-based planning as a runtime interpretation problem: the plan is not merely a document, but an executable harness specification that mediates between model outputs, filesystem state, tools, validators, and multi-agent delegation.

Discussion: Planning for code generation can be understood as a core form of *agentic harness*: a control layer that organizes how an LLM agent decomposes tasks, grounds decisions in program structure, explores alternatives at inference time, and coordinates multi-stage software engineering workflows. From this perspective, planning is a set of harness mechanisms centered on one essential question: how to decide what the agent should do next, and how to keep that decision process constrained, inspectable, and coherent across long-horizon coding tasks. Notably, planning in code generation cannot be cleanly separated from the evaluation problem. Many current conclusions about the benefits of planning depend heavily on the surrounding execution conditions, including execution environments, feedback quality, tool access, trajectory budgets, and whether the benchmark truly stresses long-range dependency management rather than localized patch generation. If execution signals are weak, revision budgets are unrealistic, or benchmarks fail to expose multi-step coordination errors, then reported planning gains may not reflect genuine improvements in agent-level problem solving. Therefore, planning is not only a method design problem, but also a harness problem between the agent and the environment. Looking forward, the central challenge is not merely to build larger planners or longer reasoning traces, but to design more reliable agentic harnesses for planning: adaptive commitment mechanisms that decide when to follow, revise, or abandon a plan; structurally meaningful planning states that expose dependencies and progress; efficient exploration-and-revision strategies that use feedback without excessive computation; and rigorous long-horizon evaluation paradigms that can faithfully measure planning quality beyond final-pass accuracy.

3.2. Memory and Context Engineering for Agent Harness

Memory has become a core infrastructure for code agents, largely because real-world software engineering tasks are inherently long-horizon and state-intensive [174, 175]. Unlike single-turn code completion, practical coding scenarios require an agent to sustain a sequence of interdependent steps across many rounds of interaction, such as requirement understanding, code localization, evidence retrieval, multi-file editing, test execution, bug fixing, and regression verification [176, 177]. This introduces a fundamental tension **between the limited context window of the model and the continuously expanding intermediate state of the task**. From a harness perspective, memory is not simply a larger context window or a vector database.

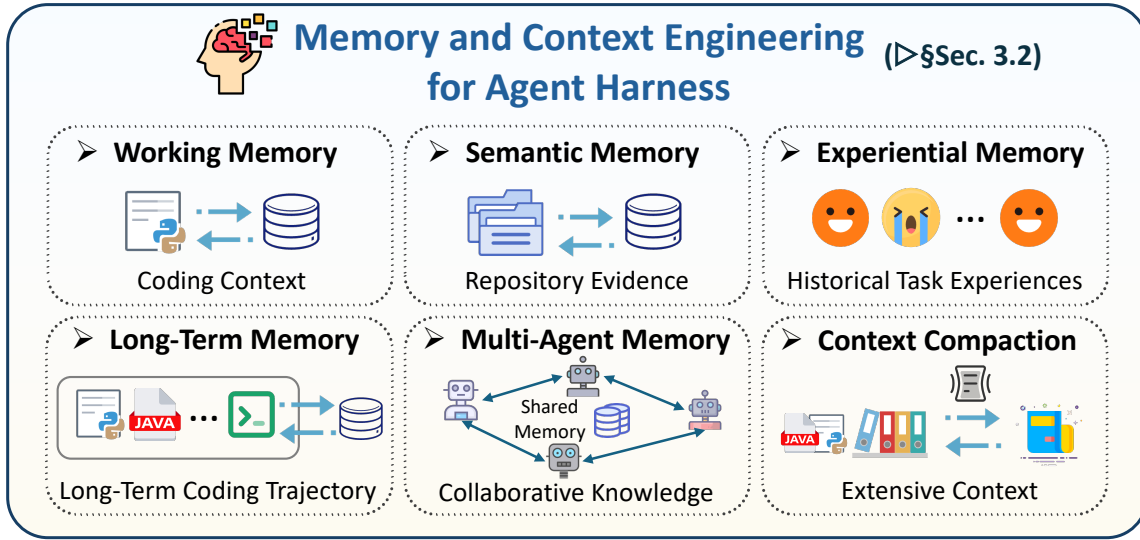


Figure 6: Overview of memory and context engineering mechanisms for agent harnesses.

It is a state-management layer that decides which information should remain in the active model context, which information should be compacted into summaries, and which information should be offloaded to durable external storage [178]. Without an effective memory mechanism and context management, an agent can easily lose critical clues during long-range reasoning, repeat searches and analyses that were already completed, or break local consistency established in earlier steps during later modifications [179, 175].

Early systems largely relied on prompts to preserve historical information, treating memory as little more than conversation history or an unstructured scratchpad. However, with the emergence of repository-level repair and other long-horizon coding tasks, it has become increasingly clear that simply accumulating natural language history cannot reliably support complex software engineering loops [180]. As a result, memory is now increasingly externalized as a system component that is retrievable, governable, and traceable. In this subsection, we categorize memory in code agents according to their **primary functional role** in the software engineering loop. Under this view, existing approaches can be broadly organized into five types: *working memory*, *semantic memory*, *experiential memory*, *long-term memory*, and *multi-agent memory*. In addition, we discuss context compaction and state offloading as cross-cutting context-engineering mechanisms that determine how large execution artifacts move between the active model context and durable task state. Representative works are illustrated in Table 5.

3.2.1. Working Memory

Working memory supports state maintenance along the current coding-task trajectory [181]. Its central concern is not how much history to retain, but which pieces of information are most useful for the next action under a limited context budget. In code agents, working memory often appears as structured prompt regions, state summaries, failed-test records, file lists, or critical stack information. Its purpose is to mitigate context explosion, reduce repeated localization, and preserve the local consistency of an ongoing repair or editing trajectory [57, 182, 183, 45]. From a harness perspective, working memory is the active control surface between the model and the code environment: it determines what the agent observes before choosing the next tool call, edit, or verification step. Representative systems such as SWE-agent [57] and RepairAgent [183] show that, even with the same underlying model, repository-level repair performance can vary substantially

Table 5: Representative memory and context management mechanisms for code-agent harnesses.

Method	Role	Managed State	Harness Operation	Primary Use
SWE-agent [57]	Working Memory	Repair trajectory; runtime state	Structured state tracking	Grounds repo repair in files, commands, and tests
CodeMem [45]	Working Memory	Context slots; edit state	Budgeted slot management	Stabilizes multi-step edits under context limits
RepairAgent [183]	Working Memory	Bug evidence; tool outputs	Dynamic prompt-state updates	Carries evidence across autonomous cycles
AutoCodeRover [46]	Semantic Memory	Repo structure; code evidence	Structure-aware retrieval	Grounds localization and patching in repo structure
RepoCoder [47]	Semantic Memory	Retrieved repo context; snippets	Iterative repo retrieval	Expands evidence for context-aware generation
CodeRAG [187]	Semantic Memory	Repo knowledge; code paths	Querying; multi-path retrieval; reranking	Selects repo knowledge for long-context completion
MemGovern [48]	Experiential Memory	Trajectories; reflections; critiques	Governed experience replay	Reuses quality experience while filtering noise
ExpeL [189]	Experiential Memory	Reflection traces; learned lessons	Reflection replay	Reuses reflections as task-solving strategies
MemCoder [190]	Long-term Memory	Commits; root causes; validated fixes	Structured memory; self-internalization	Learns repo-specific intent-to-code mappings
TALM [191]	Long-term Memory	Task histories; reasoning traces; validated code	Vector retrieval; consolidation	Reuses past episodes for tree-structured generation
MIRIX [192]	Multi-agent Memory	Cross-agent state; interaction history	Cross-agent memory routing	Routes shared memory across specialized roles
ChatDev [193]	Multi-agent Memory	Dialogue history; software artifacts	Phase-level context passing	Maintains context across role-based phases
LongCodeZip [194]	Context Compaction	Long code context; repo snippets	Coarse-to-fine compression	Compresses code while preserving reasoning cues
SWE-Pruner [195]	Context Compaction	Interaction context; surrounding code	Task-aware pruning	Removes irrelevant context before agent decisions
SWEZZE [196]	Context Compaction	Issue context; fix ingredients	Lightweight learned compression	Distills compact, fix-relevant evidence

depending on how interaction state and execution feedback are organized. CodeMem [45] similarly treats context as a managed resource, using budgeted memory slots to stabilize multi-step edits.

3.2.2. Semantic Memory

Semantic memory provides task-relevant external evidence for the current coding process [184, 175]. In code-agent settings, such evidence is usually repository-specific and program-structured, including class definitions, function implementations, call relations, configuration files, documentation, issue descriptions, dependency metadata, and historical implementation patterns. Semantic memory therefore transforms the external codebase into a queryable evidence space that the harness can retrieve from and inject into the active context [46, 185, 186, 187, 188]. Representative works such as AutoCodeRover [46] and RepoCoder [47] show that repository-level coding tasks benefit not simply from retrieving more content, but from retrieving evidence aligned with program structure. Mechanisms such as AST-based structured chunking, iterative query rewriting, and retrieval strategies conditioned on current localization clues can substantially improve the utility of retrieved context for downstream generation. In this sense, semantic memory turns the codebase into a structured evidence layer for the current decision process.

3.2.3. *Experiential Memory*

As code agents move from single-task completion toward continual repair and cross-project generalization, increasing attention has been paid to experiential or episodic memory [197, 198]. Unlike working memory, which maintains the current trajectory, or semantic memory, which retrieves repository evidence, experiential memory captures reusable experience accumulated across tasks, such as repair trajectories, failure cases, debugging records, and higher-level strategy patterns [189, 199, 200]. Its main value lies in enabling cross-task transfer. Through mechanisms such as experience cards, reflection buffers, and record-and-replay pipelines, a system can convert past successful or failed debugging processes into reusable units for future problem solving [199, 48, 201]. Works such as MemGovern [48] further suggest that the quality of stored experience matters more than its scale. Ungoverned historical records can introduce semantic noise, error propagation, and false retrievals, whereas curated and quality-controlled experiential memory is more likely to become a useful asset for repository-level repair.

3.2.4. *Long-Term Memory*

When coding trajectories become longer, working memory and semantic memory alone are insufficient, because the system must also cope with memory growth, compression-induced evidence distortion, and long-term drift. This makes long-term retrieval planning and memory control an increasingly important research direction [202, 203, 204, 205, 206]. The focus therefore shifts from memory capacity to memory governance. Representative systems such as MemGPT [207] and MemoryOS [208] move the discussion from what to store toward when to write, when to compress, when to retrieve, and how to avoid contamination. Recent code-centric studies further ground this line of work in software engineering workflows. MemCoder [190] leverages structured historical commits and human-validated solutions as persistent memory, enabling repository-specific experience accumulation over time. TALM [191] incorporates long-term memory into multi-agent code generation, retrieving prior problem–solution traces and consolidating overlapping memories to control redundancy. These works suggest that, for code agents, long-term memory should not simply accumulate more history, but preserve validated and reusable experience in a compact and controllable form. Otherwise, memory may shift from a resource for long-horizon software engineering into a burden that amplifies noise, staleness, and error.

3.2.5. *Multi-Agent Memory*

Multi-agent memory extends state management from an individual agent to a shared harness. From a systems perspective, memory in code generation has a strong collaborative dimension [209, 210]. In multi-agent frameworks, memory is not only a container for individual state, but also a medium for information sharing, intention passing, and consistency maintenance across specialized roles [211]. Representative works such as AgentCoder [50], MapCoder [44], MIRIX [192], ChatDev [193], and G-Memory [211] illustrate how memory supports multi-agent planning, testing, reviewing, and trajectory coordination. In this setting, the central challenge is no longer only retrieving relevant content, but controlling the granularity of sharing, preventing information flooding, and supporting bidirectional access between high-level decisions and fine-grained execution traces [210]. Accordingly, memory in multi-agent code generation increasingly resembles a shared blackboard or collaborative state graph rather than a purely individual storage unit [212, 213].

3.2.6. *Context Compaction and State Offloading*

Context compaction and state offloading are cross-cutting context-engineering mechanisms for memory in code-agent harnesses [214]. Their goal is not to define another memory category, but to control the

boundary between active model context and durable task state. Long-horizon software engineering workflows continuously generate high-volume artifacts, such as build logs, execution traces, repository diffs, test outputs, and intermediate plans. Directly placing these artifacts into the prompt can quickly overload the context window, amplify noise, and obscure decision-relevant evidence. A harness must therefore decide which observations should remain in the active context, which should be compacted into concise summaries, and which should be offloaded to external storage with retrievable handles [178]. Context compaction compresses long interaction histories and massive tool outputs into structured, provenance-preserving summaries. For example, a failing-test report can be reduced to the failing test name, key stack frames, suspected files, and links to the full log [196, 215, 194, 195]. State offloading complements this process by preserving full-fidelity artifacts outside the active window, such as in files, databases, trace stores, or protocol-style resource interfaces such as MCP-style servers. The agent then receives compact summaries and resource identifiers rather than raw logs or traces. By separating decision-relevant context from durable evidence, context compaction and state offloading make memory more scalable, auditable, and compatible with execution-time verification.

Discussion: Memory in code-as-agent-harness systems can be understood as a unified state-management layer that connects context management, repository evidence retrieval, experiential transfer, long-term control, and multi-agent synchronization. Rather than being a single data structure, an enlarged context window, or simply a vector database, memory coordinates where task-relevant state should reside and how it should be reused throughout long-horizon software engineering workflows. Working memory keeps the next action grounded; semantic memory exposes repository evidence; experiential memory supports cross-task transfer; long-term memory preserves validated knowledge; and multi-agent memory synchronizes shared state across roles. Context compaction and state offloading further extend this layer by separating decision-relevant active context from durable full-fidelity artifacts, making memory more scalable, auditable, and compatible with execution-time verification. Importantly, memory research in code agents cannot be separated from *evaluation reliability*. Many conclusions about memory gains depend on the quality of evaluation pipelines [216, 217]: if tests are insufficient, log parsing is flawed, or benchmarks suffer from memorization and contamination, then reported improvements may not reflect robust long-horizon behavior. Looking forward, the key challenge is not merely to enlarge memory capacity, but to build higher-quality write gates, structurally aligned retrieval keys, provenance-preserving compaction mechanisms, reliable state offloading protocols, and rigorous evaluation paradigms that measure whether memory truly helps agents remain grounded, consistent, and verifiable over extended trajectories.

3.3. Tool Use for Agent Harness

Tool usage is the action and observation layer of the code-agent harness. Once code is placed inside the agent loop, the model must not only generate text, but also search repositories, edit code, execute tests, call APIs, query documentation, and verify intermediate results [218, 219]. Tools therefore expand the agent’s action space while also exposing external feedback signals that make the harness executable and inspectable. From the perspective of code as agent harness, tool use is not merely an auxiliary capability for code generation. It is a governed interface between model intent and external systems. A reliable harness must decide which tools are available, how their schemas are exposed, what permissions each tool receives, where execution happens, how results are sanitized or compacted, and when risky actions require human approval. Recent agent SDKs and software-agent platforms make this shift explicit by packaging tools, sessions, guardrails, handoffs, workspaces, and execution environments into reusable harness components [58, 220, 221]. In parallel, sandboxed execution environments, including containerized or microVM-based workspaces, isolate agent actions from the host system and make code execution more reproducible and auditable [22, 222, 223].

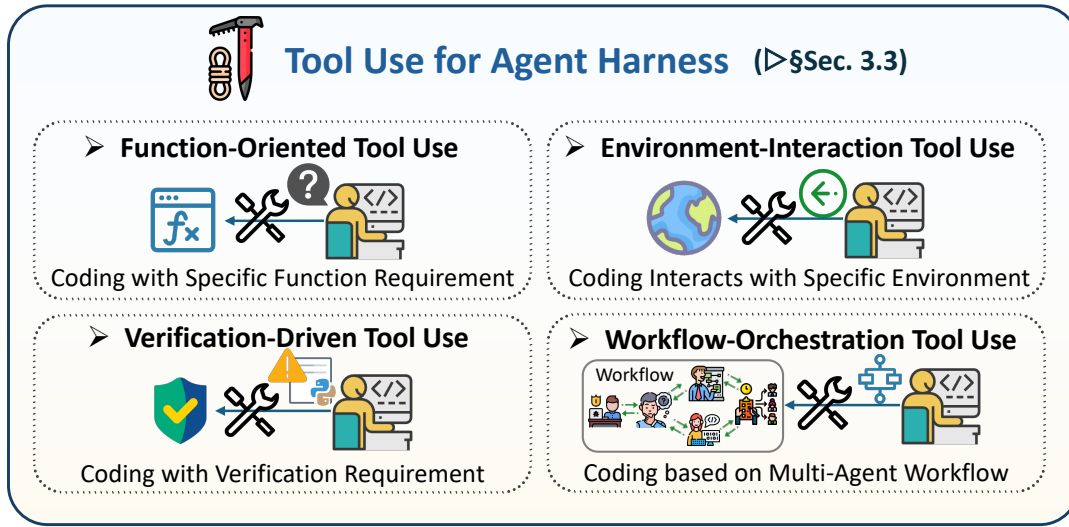


Figure 7: Overview of tool-using mechanisms for agent harnesses.

This harness-level view also highlights the importance of **tool lifecycle control**. Before a tool is executed, the harness may apply permission checks, policy rules, argument validation, or human-in-the-loop gates. After execution, the harness may sanitize outputs, summarize large logs, offload traces to durable storage, update memory, or trigger verification tools. Lifecycle hooks make these control points explicit. They turn tool use from a raw model-selected action into a monitored transition in the agent’s execution loop.

Existing work on tool usage for code agents can therefore be organized according to the primary harness function that tools serve: (1) *function-oriented tool use*, (2) *environment-interaction tool use*, (3) *verification-driven tool use*, and (4) *workflow-orchestration tool use*. Function-oriented tools ground the agent in APIs, libraries, and external documentation. Environment-interaction tools allow the agent to act inside repositories, terminals, IDEs, browsers, and sandboxes. Verification-driven tools provide deterministic feedback through tests, linters, type checkers, static analyzers, and runtime errors. Workflow-orchestration tools coordinate multiple tools, roles, memory updates, and lifecycle policies into a reliable long-horizon execution process. Representative works are illustrated in Table 6.

3.3.1. Function-Oriented Tool Use

This line of work uses tools primarily to fill gaps in the model’s programming knowledge, especially APIs, libraries, documentation, and external coding utilities [19, 224, 225, 227, 228, 229]. ToolCoder [19], for example, starts from a clear bottleneck: code models often hallucinate APIs, choose inappropriate functions, or fail on public and private libraries with sparse training coverage. To address this problem, it integrates API search tools into the code generation process and trains models to decide when to query the tool and how to select APIs from retrieved results. The key contribution is therefore not better syntax generation alone, but better knowledge acquisition and API grounding. More broadly, retrieval-oriented methods reduce dependence on parametric memory and make code generation more adaptable to long-tail APIs, private libraries, and continuously evolving software ecosystems [225, 230]. They are most effective when the main bottleneck is that the model lacks reliable knowledge of which function, API, or library construct should be used. Accordingly, the core design challenges lie in query formulation, result selection, evidence compression, and robust injection of retrieved knowledge into downstream generation. These agentic methods are particularly suitable for API-oriented generation, library migration, and private SDK

Table 6: Representative tool-use mechanisms for code-agent harnesses.

Method	Role	Tool Boundary	Harness Operation	Primary Use
ToolCoder [19]	Function-oriented	API search tools	API selection via trigger prediction	Grounds generation in retrieved APIs
CodeQA [224]	Function-oriented	API/doc query tools	Tool-augmented API QA	Retrieves API evidence for coding
RAG-for-Code [225]	Function-oriented	Repo, docs, API	Retrieval-augmented context	Knowledge for long-tail libraries
CodeAgent [185]	Environment-interaction	Repo files, tests	Repo navigation, editing, validation	Repo-level coding via environment interaction
SWE-agent [57]	Environment-interaction	Shell, editor, repo, tests	Agent–computer interface loop	Resolves GitHub issues via shell commands
AgentCoder [50]	Verification-driven	Test generation	Programmer–tester–executor loop	Refines code via generated tests
VeriGuard [226]	Verification-driven	Execution, tests, verifier	Verifier-guided tool loop	Gates and repairs code via verification
ToolNet [49]	Workflow-orchestration	APIs, tools, execution	Learned multi-tool policy routing	Routes tool invocations across workflows
MapCoder [44]	Workflow-orchestration	Coding agents	Multi-agent tool-supported workflow	Coordinates planning, generation, debugging
OpenHands [58]	Workflow-orchestration	Workspace, terminal, browser, files, runtime	Unified software-agent workspace	Long-horizon tasks via reusable interfaces

usage, but retrieval alone is often insufficient when tasks require cross-file understanding and reasoning, runtime debugging, or repository-wide dependency analysis.

3.3.2. Environment-Interaction Tool Use

Unlike function-oriented tools, environment-interaction approaches treat tools as the interface through which an agent acts inside the software engineering environment [231, 232, 233, 234]. Their central problem is no longer only to obtain missing functions, but to operate effectively over repositories, development artifacts, and execution environments. CodeAgent [185] shows that real-world repository-level code generation is not simply about completing a single function from a prompt. Instead, the model must locate relevant files, understand dependencies, inspect documentation, implement modifications, and validate outcomes through testing. To support this process, CodeAgent integrates programming tools and agent strategies for information retrieval, code-symbol navigation, code implementation, and test interaction over real repositories. SWE-agent [57] pushes this idea further by formalizing the agent-computer interface, where shell commands, file editing, and test execution become the primary interaction channel. RepairAgent [183] similarly equips the agent with repair-specific tools for reading code, searching repair ingredients, applying patches, and running tests. Together, these methods define the core trajectory of environment-interaction tool use, which is especially relevant for repository-level generation, issue resolution, and open-ended software engineering tasks.

3.3.3. Verification-Driven Tool Use

A third line of work uses tools primarily for post-generation verification and iterative improvement. Verification-driven tool use treats external tools as deterministic sensors for the harness. Compared with function-oriented and environment-interaction tools, these approaches do not necessarily emphasize external retrieval or broad repository navigation. Instead, they use tests, execution results, compiler errors, runtime traces, type checkers, static analyzers, and verifier feedback as the main signals for improving code quality [226, 235, 236, 237].

AgentCoder [50], for example, uses a programmer agent, a test designer agent, and a test executor agent to form a closed loop of code generation, test construction, execution, and refinement. In this paradigm, the central role of tools is verification rather than retrieval. From the code-as-agent-harness view, verification tools make agent progress inspectable: test failures, stack traces, coverage gaps, type errors, and static-analysis warnings become structured observations that update working memory and guide the next action. The key design issue is how to route these observations back into the loop [226]. Since raw logs may be too long or noisy for the active context, the harness should parse, summarize, and offload verification traces while preserving full-fidelity artifacts for audit and replay.

3.3.4. Workflow-Orchestration Tool Use

Workflow-orchestration tool use focuses on how multiple tools, roles, and control policies are organized into a coherent agent workflow [238, 239, 240, 241]. In long-horizon software tasks, the agent may need to retrieve evidence, localize bugs, modify files, run tests, inspect failures, update memory, ask for approval, and repeat this cycle several times. The challenge is not simply adding more tools, but deciding when each tool should be invoked, with what permissions, under which context, and how its result should update the harness state [49]. Recent agent SDKs and software-agent platforms make this orchestration layer explicit by packaging typed tool schemas, session state, workspaces, guardrails, handoffs, tracing, and human-review mechanisms into reusable harness components. Lifecycle hooks further refine this boundary: pre-use hooks can validate arguments, enforce permission policies, or block risky commands, while post-use hooks can sanitize outputs, compact logs, update memory, or trigger follow-up verification. Representative systems such as MapCoder [44] exemplify workflow orchestration by assigning agents to example recall, planning, code generation, and debugging, thereby decomposing a difficult coding problem into coordinated subproblems. CodeAgent [185] also studies how tool calls should be scheduled and structured in repository-level workflows. This class is particularly important for long-horizon code agents, where realistic software tasks require demand decomposition, context selection, candidate exploration, execution-based verification, and final repair under explicit control policies [49, 242].

Discussion: Tool usage in code agents has evolved from isolated API retrieval to a full harness mechanism for action, observation, verification, and governance. Function-oriented tools ground implementation choices in external knowledge; environment-interaction tools allow agents to act over repositories and execution environments; verification-driven tools provide deterministic feedback; and workflow-orchestration tools coordinate these capabilities through SDKs, sandboxes, guardrails, and lifecycle hooks. The core challenge is no longer whether a model can call a tool, but whether the harness can make tool use safe, auditable, and useful for long-horizon execution. Future code-agent harnesses should support typed tool schemas, permission-aware invocation, sandboxed execution, lifecycle hooks, result sanitization, context compaction, state offloading, and reproducible traces. These mechanisms ensure that tools expand the agent’s action space without sacrificing reliability, safety, or verifiability.

3.4. Harness Control through the Plan, Execute, and Verify Loop

Code-as-harness systems require a control loop that turns model intentions into bounded, observable, and revisable state transitions. This subsection frames that loop as *Plan–Execute–Verify* (PEV): the harness first externalizes an intended change and its validation criteria, then executes the change inside a sandboxed and permissioned environment, and finally verifies the resulting state through deterministic sensors and human-review gates. This framing unifies planning, execution, debugging, verification, and escalation as parts of a single harness-level control process.

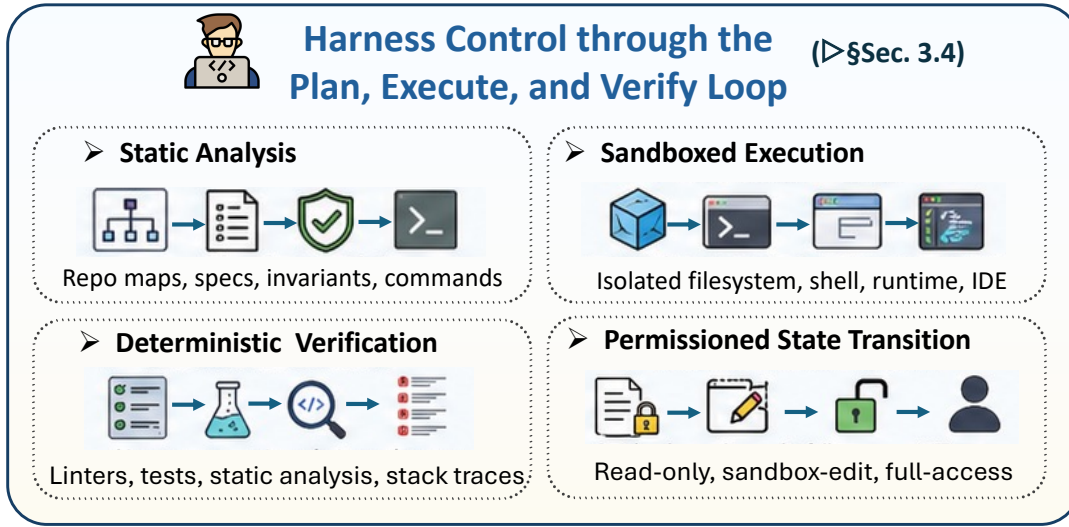


Figure 8: Overview of harness control through PEV loop.

3.4.1. From Debugging to Harness-Level Control

The preceding subsections describe planning as trajectory control, memory as state management, and tool use as a governed action interface. Feedback-guided debugging connects these mechanisms into a closed loop: plans specify intended changes, memory preserves relevant evidence, tools execute and inspect actions, and validation signals determine whether the agent should continue, revise, or stop. As code-centric agents move from single-turn generation to repository-level software work, debugging is therefore better understood as control over executable program state rather than as a post hoc correction stage. Generated programs can fail through syntax errors, runtime exceptions, incorrect outputs, incomplete edge-case handling, unsafe operations, or violations of project-specific conventions, making one-pass generation insufficient [243]. Recent systems revise code through feedback from compilers, runtimes, tests, static analyzers, humans, and auxiliary agents [244, 245, 246, 23]. From the harness perspective, this process can be reframed as a *Plan–Execute–Verify* (PEV) loop: the agent externalizes an intended trajectory, executes bounded actions inside a controlled environment, and verifies the resulting state before the next transition. The growing engineering ecosystem around agent harnesses reinforces this view: recent curated resources distinguish orchestration, working state, execution substrates, evaluation harnesses, observability, and governance as separable harness layers rather than incidental implementation details [247, 248, 249, 25].

In this view, the harness acts as a *cybernetic governor*: a control layer that observes the effects of agent actions and regulates subsequent state transitions. Rather than merely forwarding error messages to the model, it observes the repository and execution environment through deterministic sensors such as linters, parsers, compilers, type checkers, unit tests, integration tests, static analyzers, fuzzers, runtime monitors, and CI pipelines. These sensors turn a coding trajectory into inspectable signals, including pass/fail outcomes, diagnostics, failing traces, coverage gaps, security warnings, resource limits, and policy violations. The harness can then decide whether to continue execution, revise a patch, request more context, route the task to another module, reduce permissions, or escalate to a human reviewer. Table 7 summarizes this control surface; the remainder of this subsection follows the loop from contract formation, through sandboxed state transition, to deterministic verification and evidence-grounded repair.

Table 7: Representative methods and systems for PEV-loop harness control.

Method	PEV Role	Core Mechanism	Signals and Gates
CodePlan [42]	Plan, structural	Dependency plan graph	Repo links, critiques
MapCoder [44]	Plan, orchestration	Map-code-test stages	Handoffs, tests, failures
OpenHands [250]	Full PEV harness	Stateful edit-exec workspace	Diffs, logs, tests, approvals
SWE-agent [57]	Execute, CLI	Replayable shell interface	Commands, patches, tests
Daytona [251]	Execute, cloud sandbox	Isolated dev workspace	Files, limits, snapshots
E2B [252]	Execute, code-browser sandbox	Cloud code-browser sandbox	Stdout, limits, UI state
Self-Debugging [243]	Verify, self-debug	Explanation-guided repair	Errors, tests
Reflexion [244]	Verify, reflection memory	Verbal feedback memory	Outcomes, critiques
Debug Like a Human [245]	Verify, stepwise debug	Runtime-step checks	Traces, variables, asserts
Iterative Refinement [246]	Plan-Verify feedback	Project-context repair	Compiler diagnostics
QualityFlow [253]	Verify, quality gate	Quality feedback routing	Tests, success, stopping
AgentCoder [50]	Verify, multi-agent repair	Coder-tester-executor loop	Tests, failures, critique
AutoSafeCoder [52]	Verify, safety sensors	Static checks, fuzzing	Alerts, traces, tests
VeriGuard [226]	Verify, verified gen.	Verifier guard layer	Proofs, tests, alerts
LiteLLM [254]	Permission gateway	Proxy policy routing	Approvals, denials, cost logs

3.4.2. Planning as Contract Formation

The planning phase turns a user request into an explicit contract over the next state transition. A robust plan does more than decompose the request into implementation steps; it also identifies relevant files, expected invariants, validation commands, rollback points, and risky operations. This makes planning a harness artifact rather than an unobserved reasoning trace. In repository-level tasks, such artifacts constrain the subsequent action space by specifying which components may be read, which files may be edited, and which verification criteria must be satisfied before completion [40, 42, 44]. Repository-local instructions and tool protocols strengthen this contract layer: AGENTS.md-style guidance, MCP server registries, typed tool schemas, adapters, and protocol gateways make the available actions inspectable before execution rather than discovered opportunistically during execution [255, 256, 257, 258, 259, 260, 261, 262]. The PEV framing also clarifies why planning and debugging should not be separated: failed verification updates the plan, while the plan determines which verification evidence is meaningful.

3.4.3. Sandboxed Execution and Permissioned State Transition

The execution phase realizes the plan as a bounded and observable state transition. The sandboxed environment is the operational substrate of the loop: it provides an isolated filesystem, dependency state, shell, language runtime, browser or IDE interface, and resource boundary in which agent-generated actions can be run without directly compromising the host system [263, 22]. Contemporary execution-substrate work is best read as functional clusters rather than as an undifferentiated catalog. Coding sandboxes expose filesystems, Git operations, shells, package managers, and code-execution backends [251, 252, 264, 265, 266, 250]; computer-use substrates add browser, desktop, LSP, or IDE state [267, 268, 269, 270, 271]; and durable runtimes emphasize microVM or WASM isolation, snapshots, warm pools, resumable sessions, benchmark environments, and always-on operating contexts [272, 273, 274, 275, 276, 277, 278]. Sandboxes also improve reproducibility because the harness can replay the same patch, command, seed, dependency lockfile, or test configuration under comparable conditions. Without this stable substrate, verification signals become

difficult to interpret, and failures may reflect environment drift rather than program defects [250, 217, 279].

Execution must also be permissioned. A multi-tier model separates low-risk observation from high-risk action: a read-only tier supports repository browsing, retrieval, static inspection, and log analysis; a sandbox-edit tier supports local patching, test execution, and temporary dependency installation inside an isolated workspace; and a full-access tier covers network access, credentials, deployment commands, package publishing, destructive filesystem operations, or Git history mutation. Actions in the final tier should be guarded by mandatory human-in-the-loop (HITL) gates because their consequences can extend beyond the sandbox. Recent software-agent systems and harness engineering work increasingly expose these control points through explicit tools, sessions, policies, approval prompts, and audit logs [280, 250, 281, 178, 282, 283]. Gateway and policy layers then provide the production counterpart: systems for model routing, tool registration, proxy-level logging, centralized guardrails, security automation, and falsifiable approval evidence keep governance outside the prompt alone [254, 284, 285, 262, 286, 287, 288, 289, 290, 291].

3.4.4. Verification through Deterministic Sensors

The verification phase closes and, when necessary, reopens the loop by comparing the new state against explicit constraints. Compilation and static-analysis feedback provide low-cost sensors before full execution, including parser diagnostics, type errors, lint warnings, and security alerts [246, 292, 293]. Runtime signals expose failures that only arise along concrete execution paths, such as exceptions, assertion breaks, invalid API usage, resource exhaustion, and timeouts [294, 295, 245]. Test-based feedback then evaluates whether the observed behavior satisfies the intended specification, using unit tests, integration tests, regression tests, fuzzing, or benchmark-specific evaluators [243, 296, 297, 298]. Evaluation harnesses broaden this idea from a single test command to repeatable task distributions: they encode evaluator logic, simulation hooks, red-team cases, or RL-style environments that can compare harness variants under controlled conditions [299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309]. Compared with natural-language critique, these sensors are deterministic or at least reproducible enough to serve as control signals. Human or agentic critiques remain useful when failure evidence is sparse, but in a governed PEV loop they should interpret sensor outputs rather than replace them [244, 310, 54].

Verification also supplies the evidence for repair, reflection, and termination, so these activities are treated as consequences of the Verify phase rather than as an independent stage. When a check fails, the same sensor evidence can determine whether the harness should ask the model to diagnose the failure, retrieve missing context, regenerate a localized patch, route the task to a testing or security agent, or abandon the current branch. Self-reflection mechanisms help transform raw diagnostics into actionable hypotheses, such as whether the failure comes from incorrect control flow, missing edge cases, misunderstood APIs, or inadequate tests [311, 166]. However, reflection is reliable only when it remains grounded in executable evidence. Systems such as AgentCoder, AutoSafeCoder, and QualityFlow illustrate this principle by combining agentic critique with independent execution, static analysis, fuzzing, or test-quality gates [50, 52, 253]. Termination should likewise be governed by verification rather than by model confidence: a loop can stop when required checks pass, when additional attempts no longer improve the state, when the risk tier changes, or when human review is required.

Discussion: Recasting iterative debugging as the PEV loop emphasizes that reliability comes from governed state transitions, not simply from better repair prompts. Planning externalizes intended changes and risk assumptions; execution applies them inside sandboxed and permissioned environments; verification uses deterministic sensors to decide whether the state is acceptable; and HITL gates preserve accountability when the action space crosses a safety boundary. This framing unifies static analysis, runtime errors, tests, critique,

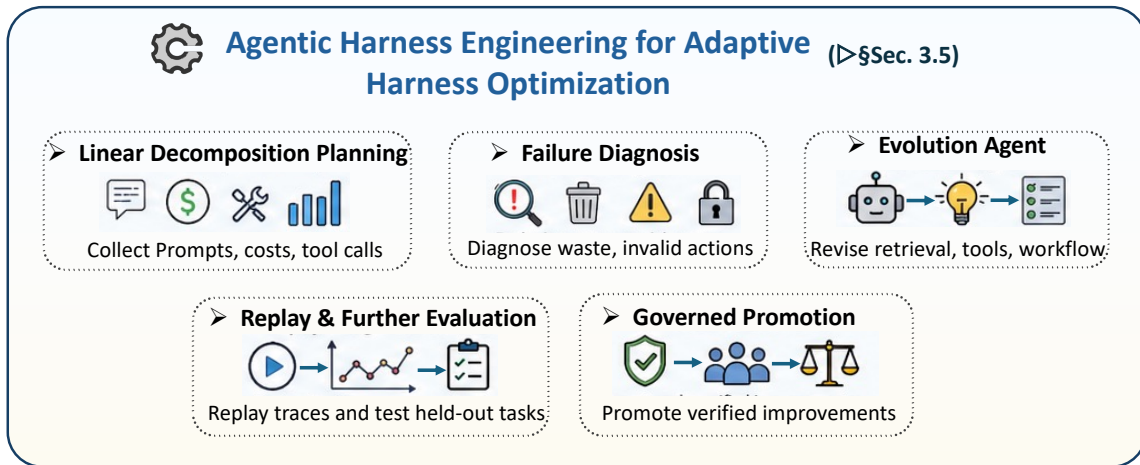


Figure 9: Overview of harness engineering for adaptive harness optimization.

self-reflection, and human review as components of a cybernetic harness that regulates the agent’s trajectory over executable program state.

3.5. Agentic Harness Engineering for Adaptive Harness Optimization

Agentic Harness Engineering (AHE) names a harness-level design problem: how to measure and revise the software substrate that turns a language model into a coding agent. Whereas prompt engineering changes instructions and context engineering changes what evidence is presented to the model, AHE treats the operating environment itself as the object of analysis, including tool schemas, planning artifacts, memory policies, retrieval strategies, sandbox configuration, verification sensors, permission tiers, routing rules, multi-agent workflows, and human-review gates [281, 178]. This perspective is useful because many observed failures in code agents arise from missing repository context, brittle tool interfaces, weak validators, excessive token cost, poor retry policies, or mismatched permission boundaries rather than from model generation.

Existing work can be read as three complementary strands. AutoHarness studies automatic synthesis of code harnesses [14]; Meta-Harness formulates harness design as an optimization problem over model-facing infrastructure [13]; and observability-driven AHE emphasizes telemetry-rich diagnosis of where the agent loop fails and which harness component should change [281]. Related work on reflective prompt evolution, self-evolving workflows, and live software-engineering agents supports the same systems view: changing the scaffold around the model can change agent behavior without retraining the base model [18, 312, 182]. Engineering guides from OpenAI, Anthropic, and LangChain converge on the same practical lesson: reliable agents require explicit harness loops, tool contracts, trace replay, evaluation suites, context budgets, and controlled execution boundaries [248, 249, 313, 314, 28].

3.5.1. Deep Telemetry as the Optimization Substrate

The central substrate of AHE is *deep telemetry*: structured traces that connect model decisions, harness actions, environment states, and outcomes. A shallow log may record only the final answer or pass/fail result. Deep telemetry records the decision process in greater detail: prompts and retrieved context, token usage and cost, model/tool latency, tool arguments, permission requests, edited files, sandbox snapshots, command outputs, test results, stack traces, lint warnings, branch decisions, rejected alternatives, human interventions, and final

task outcome. In code-centric settings, these traces are especially valuable because program execution already exposes state transitions through logs, tests, diffs, and runtime behavior [128, 96, 37]. In production systems, this role is increasingly served by observability and reliability stacks that record traces, metrics, prompts, model traffic, eval results, and cost signals [315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327]. Evaluation, observability, and governance systems therefore provide complementary telemetry channels: evaluators expose task-level regressions, tracing stacks expose trajectory-level causes, and policy gateways expose boundary violations that an Evolution Agent can turn into harness revisions.

Telemetry turns harness revision from anecdotal debugging into comparative diagnosis. Token-cost traces reveal when retrieval or reflection stages consume budget without improving verification outcomes. Decision-tree traces show where the agent repeatedly chooses unproductive tools, edits irrelevant files, or loops between failed strategies. Failure traces cluster recurring patterns such as missing dependencies, weak tests, hallucinated APIs, flaky sandboxes, over-permissive tool calls, or premature termination. Because these signals are linked to concrete artifacts, they can be replayed and compared across harness versions, making it possible to evaluate whether a change improves reliability rather than merely changing surface behavior [216, 217].

3.5.2. The Evolution Agent

An *Evolution Agent* is a meta-level agent that uses deep telemetry to propose, evaluate, and promote revisions to harness components. Unlike a task agent, which edits the target repository, the Evolution Agent edits the operating conditions under which later task agents work. Its input is a corpus of trajectories; its output may be a revised prompt template, a retrieval policy, a more precise tool schema, an added validator, a changed permission rule, a workflow-topology adjustment, or a new regression test. This role is closely related to self-evolving multi-agent systems in which specialized agents inspect execution logs, attribute failures to workflow components, and update collaboration structures [328, 329]. In the harness setting, the same idea is generalized from multi-agent topology to the control surface of the agent runtime.

A typical Evolution-Agent loop contains five stages. First, it *observes* trajectories by collecting telemetry from PEV executions. Second, it *diagnoses* failure modes by attributing cost, latency, invalid actions, test failures, or permission denials to specific harness components. Third, it *proposes* candidate revisions, such as rewriting tool descriptions, changing context packing rules, adding a linter, modifying retry limits, or inserting a HITL gate before risky commands. Fourth, it *evaluates* the revised harness on held-out tasks or replayed traces using deterministic sensors and regression tests. Finally, it *promotes* only changes that improve reliability, cost, or safety without regressing previously solved cases. This keeps AHE within the same engineering discipline as the PEV loop: proposed changes must be executed, verified, and made auditable before adoption.

3.5.3. Governed Harness Mutation

AHE should not be confused with unconstrained self-modification. Because the Evolution Agent changes the harness that controls later task agents, its actions require stronger governance than ordinary code repair. Candidate harness changes should be evaluated inside sandboxes, compared against fixed regression suites, and recorded with auditable rationales. Changes that alter permission boundaries, network access, credential handling, deployment behavior, or human-review requirements should require HITL approval before activation. In this sense, the Evolution Agent is itself subject to the PEV loop: it plans a harness mutation, executes it in an isolated evaluation environment, verifies the result through telemetry and regression tests, and escalates risky changes to humans.

Discussion: Agentic Harness Engineering extends the code-as-harness view from operating agents to analyzing

Table 8: Representative methods for Agentic Harness Engineering with telemetry-driven revision targets.

Method	Category	Telemetry	Revision Target
AutoHarness [14]	Harness synthesis	Failures, fixtures, assertions	Harness code and tests
Meta-Harness [13]	Harness search	Code, scores, traces	Prompts, tools, scripts
AHE [281]	Telemetry-driven optimization	Cost, decisions, latency, failures	Context, tools, validators
GEPA [18]	Reflective prompt evolution	Scores, feedback, critiques	Prompts and instructions
EvoMAC [328]	Workflow topology evolution	Handoffs, idle roles, loops	Agent roles and graph
SEW [312]	Self-evolving workflow	Workflow scores, failures	Stage order and roles
Live-SWE [182]	Online agent evolution	Live issue trajectories	Policies, tools, memory
GroundedTTA [232]	Test-time adaptation	State-action evidence	Adaptation rules
RLEF [104]	Execution-feedback learning	Execution rewards, failures	Feedback reward signal
DeepEval [300]	Evaluation harness	Scenario and metric traces	Regression suites, gates
FeedbackEval [23]	Repair evaluation benchmark	Feedback-task scores	Failure taxonomy and eval set
Langfuse [315]	Observability platform	Spans, cost, latency, evals	Dashboards and replay
OpenLLMetry [321]	Trace instrumentation	OpenTelemetry spans, calls	Harness instrumentation
Promptfoo [299]	Evaluation harness	Scores, regressions, failures	Eval gates and red tests
LiteLLM [254]	Gateway governance	Routing, budgets, failures	Budgets, fallbacks, tiers

the infrastructure that operates them. Deep telemetry provides evidence for locating failures across prompts, tools, memory, sandboxes, validators, permissions, and workflows. Evolution Agents use this evidence to propose and evaluate harness mutations, turning harness design into an iterative and measurable engineering process governed by verification and human approval.

4. Scaling the Harness: Multi-Agent Orchestration over Code

As AI systems tackle increasingly complex problems from function-level code synthesis to repository-level system engineering, fundamental limitations for single-agent emerge: (1) context window constraints prevent a single agent from holding an entire codebase, long interaction history, and execution trace in working memory; (2) specialization requirements make it inefficient to use one generalist agent for planning, synthesis, testing, review, and debugging simultaneously; and (3) the absence of independent coordination and verification channels prevents the agent from reliably detecting and correcting its own errors during long-horizon execution. Multi-agent systems introduce a powerful principle: once these responsibilities are distributed across specialized roles, the agent harness itself becomes more modular, inspectable, and adaptable. Early systems such as ChatDev [330], MetaGPT [55], and AgentCoder [50] demonstrate this shift by dividing software-development responsibilities among distinct agents such as architect, programmer, tester, reviewer, and executor. Coordinated through structured communication protocols and shared code artifacts, these role-specialized agents turn code from a mere output target into the shared substrate through which the overall harness plans, acts, verifies, and improves itself.

In this section, we systematically survey the rapidly growing direction on using MAS to scale coding harnesses, and we propose a new position on building shared code-centric harness substrates for AI agents.

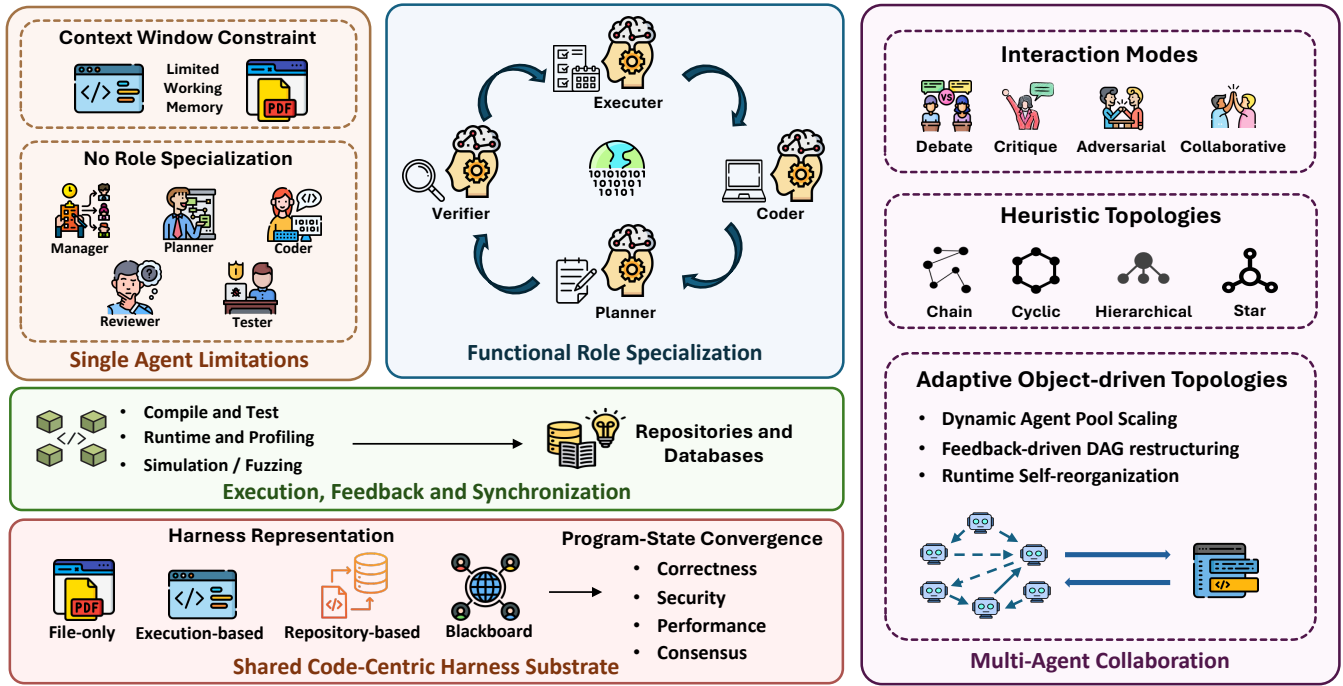


Figure 10: Overview of scaling the agent harness through multi-agent orchestration over code. The figure illustrates how role-specialized agents, shared code-centric substrates, execution feedback, and adaptive collaboration topologies address single-agent limitations in context, specialization, and self-correction.

4.1. Improved Coding Support through Multi-agent Collaboration

The most immediate contribution of multi-agent systems is that they improve coding support by decomposing the harness into specialized but coordinated components. Instead of integrating planning, synthesis, execution, and verification into a single agent loop, these systems distribute responsibility across roles that interact through shared code artifacts and feedback signals. This division of labor makes the overall harness more capable of handling complex software tasks, while also making its internal workflow more inspectable and controllable. In practice, this improvement is realized through three closely related design dimensions: how roles are specialized, how agents interact over shared program artifacts, and how the workflow topology organizes their collaboration.

4.1.1. Functional Role Specialization and Human-Guided Planning

In human software development, different roles specialize in different aspects of the development process. Many MAS naturally mirror this division of labor by assigning distinct functional roles to different agents. This specialization allows each agent to focus on a specific slice of the shared code harness, leveraging its unique capabilities and perspectives to contribute to the overall task. Here, we elaborate on the most common functional roles identified across the surveyed literature, noting that many systems implement multiple roles and that the boundaries between them can be fluid.

Program synthesis agents Program synthesis agents are responsible for generating or transforming code. They consume specifications, plans, or feedback signals and produce or revise code artifacts. This is the most

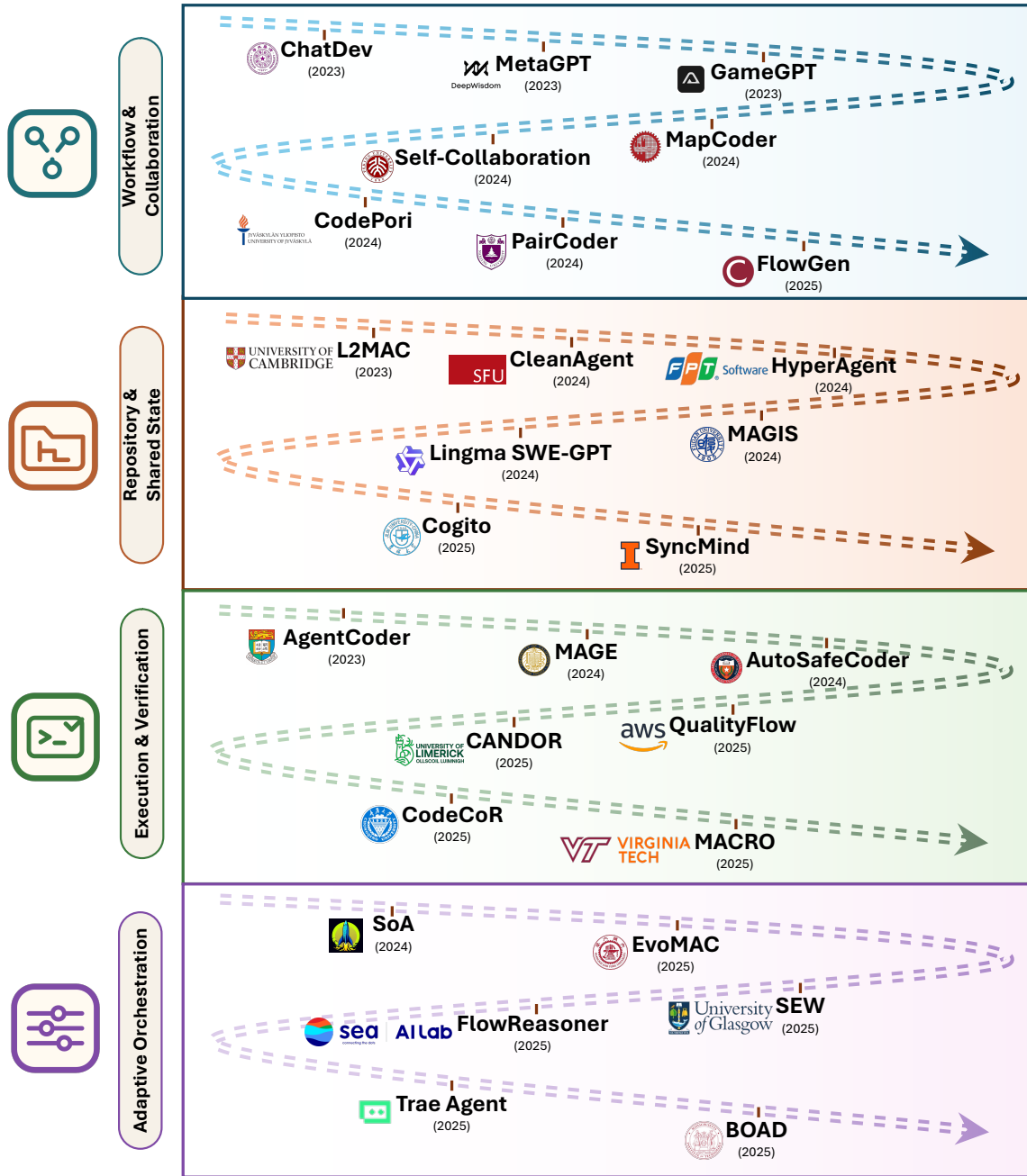


Figure 11: Roadmap of scaling code harnesses for multi-agent orchestration, organized by workflow collaboration, shared repository state, execution verification, and adaptive coordination.

common role across surveyed systems. Representative instances include the **Coder** in Self-Collaboration [56], the **Programmer** in AgentCoder [50], the **Engineer** in MetaGPT [55], the **Developer** in ChatDev [330], and the **RTL Generation Agent** in MAGE [339].

Program understanding agents Program understanding agents analyze existing code or specifications to produce higher-level representations. They own the interpretation of what the code means rather than what

Table 9: Representative MAS collaboration designs by role specialization and interaction structure.

System	Harness Substrate	Agent Roles	Interaction Mode	Topology
Self-Collaboration [56]	Blackboard, implicit	Plan, Synth., Verif. (simulated)	Critique-repair	Pre-defined cyclic
CodePori [331]	Implicit	Plan, Synth., Verif.	Collab-Synth., critique-repair	Pre-defined chain, cyclic
MAGIS [332]	Repository, evolution memory	Plan, Understand, Synth., Verif.	Critique-repair, debate, delegation	Hierarchical, cyclic, dynamic pool
HyperAgent [333]	Repository, execution	Plan, Understand, Synth., Exec	Critique-repair	Pre-defined hierarchical, cyclic
PairCoder [334]	Execution	Plan-Understand, Synth-Exec	Collab-Synth., critique-repair	Pre-defined cyclic with conditional branch
FlowGen [335]	Execution, implicit	Plan, Understand, Synth., Verif.	Critique-repair, debate	Pre-defined chain, cyclic (Scrum)
Trae Agent [336]	Repository, execution	Generate, Prune, Select	Collab-Synth., search (selection)	Pre-defined search pipeline
BOAD [337]	Repository, execution	Orchestrate, Localize, Edit, Validate	Delegation, adaptive selection	Adaptive hierarchical
FlowReasoner [338]	Execution, implicit	Meta-design, Solve	Runtime workflow generation	Objective-driven adaptive
ChatDev [330]	Implicit, borderline exec	Plan, Synth., Verif., Exec	Critique-repair, debate	Pre-defined chain (waterfall)
MetaGPT [55]	Implicit, partial blackboard	Plan $\times$ 3, Synth., Verif.	Critique-repair, pub-sub scheduling	Pre-defined chain (waterfall)
GameGPT [210]	Blackboard (dual collaboration)	Plan, Synth., Verif.	Critique-repair, collaborative	Pre-defined

it does. This category includes the Repository Custodian in MAGIS [332], the Navigator in HyperAgent [333], the RepoUer in Lingma SWE-GPT [340], and the Column-type Annotator in CleanAgent [341].

Verification agents Verification agents evaluate code quality, typically by generating test cases, running static analysis, or simulating execution. The Test Designer in AgentCoder [50] generates test cases independently of the code to avoid circular reasoning, a design principle against the mode-collapse problem where an agent’s biased tests pass its own buggy code. The Test Quality Checker in QualityFlow [253] addresses this at a meta-level, filtering synthesized tests before they are used as feedback. The Static Analyzer and Fuzzing Agent in AutoSafeCoder [52] provide security-oriented verification through static CWE analysis and dynamic crash detection, respectively. The Panelists in CANDOR [342] independently audit oracle correctness against natural language specifications rather than against the code itself, deliberately avoiding contamination by faulty implementations.

Execution agents Execution agents interface directly with the program runtime. Critically, the Test Executor in AgentCoder [50] is a deterministic Python script (not an LLM) which cleanly separates reasoning from execution and grounds the feedback signal in objective program behavior. The Executor in HyperAgent [333] runs unit and integration tests via an interactive bash shell. The Judge Agent in MAGE [339] interfaces with RTL simulation tools to produce per-clock-edge waveform snapshots.

Planning agents Planning agents decompose the overall software-development task into subtasks and assign them to synthesis agents. The Architect and Project Manager in MetaGPT [55], the Manager in MAGIS [332], the Scrum Master in FlowGen [335], and the Mother agents in SoA [212] all perform task decomposition. The Mother agents in SoA [212] are particularly notable: they dynamically spawn Child agents at runtime based on the inferred complexity of each subfunction, making planning and agent initialization interdependent.

A distinctive feature of EvoMAC [328] is the introduction of two novel meta-roles not present in any other surveyed system: the Gradient Agent, which reads execution logs to identify which agents caused failures, and the Updating Agent, which revises agent prompts and restructures the workflow DAG accordingly. These roles operate at the level of the MAS itself rather than the program, enabling the system to adapt its own structure in response to execution feedback.

4.1.2. Diverse Interaction Modes Grounded in Shared Program State

Unlike general MAS where agent interaction is primarily message-passing, code-centric interaction is characterized by artifact-mediated communication: agents observe and modify shared code artifacts, and their interaction is grounded in the objective state exposed by those artifacts and their execution results. These coordination channels are broader than source code alone: agents communicate through APIs, files, diffs, tests, logs, schemas, blackboards, and explicit workflow states. In most systems, these channels are part of the human-designed harness, while agents dynamically write to or modify the artifacts circulating within them. We identify four interaction modes.

Collaborative synthesis Collaborative synthesis occurs when two agents jointly construct a program component, analogous to pair programming [343]. The Navigator–Driver pairing in PairCoder [334] is the most direct instantiation: the Navigator generates and selects solution plans while the Driver implements them, with bidirectional information flow. CodePori [331] implements collaborative synthesis between Dev_01 and Dev_02, who exchange code drafts across three rounds. This mode is relatively rare among the surveyed system, as most systems prefer a sequential handoff rather than true co-construction.

Critique and repair Critique and repair is the dominant interaction mode across the surveyed literature. A verification or evaluation agent inspects a code artifact and produces structured feedback; a synthesis agent then revises the artifact in response. This pattern appears in some form in virtually every surveyed system. Its key design decisions are: (a) whether the critique is LLM-simulated or execution-grounded (Self-Collaboration [56] uses a simulated LLM tester, while AgentCoder [50] uses a real Python executor); (b) the richness of the feedback signal (ranging from binary pass/fail in SEW [312] to structured execution logs enumerating satisfied requirements, function errors, and unmet requirements in EvoMAC [328]); and (c) the number of repair iterations permitted before fallback.

Adversarial validation Adversarial validation is a more active form of verification in which one agent attempts to break the code through adversarial inputs, rather than passively reviewing it. AutoSafeCoder [52] implements this via its Fuzzing Agent, which generates crash-inducing input seeds using type-aware mutation and executes the code to produce crash traces. This mode has a fundamentally different character from critique-and-repair: the fuzzer does not explain what is wrong, but demonstrates a concrete execution failure, a counterexample that the coding agent must address. MAGE [339] similarly uses simulation mismatch as an adversarial signal: the Debug Agent receives the exact waveform window around the first clock-edge failure, enabling targeted repair.

Reasoning debate Reasoning debate involves agents arguing over the correctness of a decision or the interpretation of a specification, before arriving at a consensus. ChatDev [330] introduces communicative de-hallucination, a mechanism in which the assistant agent reverses roles to ask clarifying questions before committing to a response. The Scrum sprint meetings in FlowGen [335] enable disordered multi-agent discussion around a shared context buffer before the Scrum Master synthesizes a decision. CANDOR [342] implements the most structured debate mechanism: three independent Panelists evaluate oracle correctness, and a Curator aggregates their verdicts via majority vote. The kick-off meeting in MAGIS [332] involves a circular speech among the Manager and all Developer agents to negotiate task dependencies and prevent conflicts.

4.1.3. Optimized Workflow Topology for Agentic Coordination

The topology of agent interaction, who communicates with whom, in what order, and how many times, is one of the most consequential design decisions in a MAS for code generation. We organize topologies along two primary axes.

Pre-defined Heuristic Topologies The majority of surveyed systems use topologies that mirror established software development life cycle (SDLC) models. These topologies are fixed at design time and do not change in response to task complexity or system performance.

Chain (Waterfall) topologies sequence agents in a strict linear order, with artifacts flowing unidirectionally from planning to synthesis to verification. ChatDev [330] and MetaGPT [55] are canonical examples, explicitly modeling the waterfall SDLC: design → coding → testing. FlowGen [335] operationalizes three SDLC models as distinct topologies: FlowWater (strict waterfall chain), FlowTDD (requirement → design → test → implementation → fix, a test-driven reordering), and FlowScrum (cyclic iterative sprints). This paper is unique in directly comparing the implications of different SDLC-mirroring topologies for code quality. L2MAC [344] also follows a chain topology but with a novel twist: each step in the instruction plan is executed by a fresh-context agent, making the chain a sequence of independent LLM invocations sharing only the external file store.

Cyclic (Agile/Iterative) topologies introduce back-edges that allow code to be revised in response to verification feedback. AgentCoder [50] implements a programmer → test executor → (if fail) → programmer cycle, bounded at 5 iterations. Self-Collaboration [56] embeds a coder ↔ tester back-edge within its waterfall chain, max 4 iterations. PairCoder [334] enhances the cyclic pattern with multi-plan exploration: a pool of n solution plans is pre-generated via k-means++ clustering for diversity, and the cycle can switch to the next candidate plan when dead-end is detected through history-based loop analysis. MAGE [339] combines a linear initialization chain with a cyclic debug-judge loop, and introduces high-temperature candidate sampling to explore multiple program variants simultaneously.

Hierarchical topologies place one or more manager agents above a pool of worker agents, enabling decomposition-and-delegation patterns. MAGIS [332] has a Manager that dynamically instantiates one Developer agent per candidate file at runtime; each Developer edits its assigned file and reports back to the manager-review layer. HyperAgent [333] uses a planner above multiple repository navigation and editing workers, combining top-down decomposition with bottom-up repository evidence. SoA [212] pushes this hierarchy further by allowing Mother agents to spawn Child agents recursively according to inferred subtask complexity. These systems treat harness orchestration itself as a resource-allocation problem.

Star topologies center on a hub agent that coordinates multiple parallel worker agents. The CANDOR [342] Stage 3 panel is an example: a Requirement Engineer fans out to three independent Panelist+Interpreter pipelines, and the Curator aggregates their outputs. MetaGPT [55]’s publish-subscribe message pool creates a de facto star topology where the shared pool serves as the hub.

Objective-driven and Adaptive Topologies A smaller but rapidly growing class of systems treats the topology itself as a design variable to be optimized toward a code quality signal. Recent systems such as FlowReasoner [338] and BOAD [337] further reinforce this trend by treating multi-agent organization itself as an adaptive object to be generated, searched, or optimized per task.

Dynamic agent pool scaling is the simplest form of adaptivity: the number of agents scales with task complexity, but the topology type is fixed. SoA [212] implements this via a hierarchical tree of Mother and Child agents, where Mother agents decide at runtime how many subfunctions to decompose into, spawning corresponding Child agents. The key insight is that each agent’s context window remains bounded, as complexity is handled by growing the agent pool rather than growing individual context windows. MAGIS [332] similarly instantiates Developer agents dynamically based on the number of candidate files identified during repository analysis. BOAD [337] extends this line of thought from dynamic scaling to hierarchy discovery: instead of manually fixing the specialized sub-agent structure, it formulates the selection of helpful localization, editing, and validation sub-agents as a bandit-optimization problem, showing that automatically discovered hierarchical teams can outperform manually designed ones.

Feedback-driven DAG restructuring is best represented by EvoMAC [328]. Its workflow is a DAG whose nodes correspond to agents and whose edges define information flow. After each iteration, a Gradient Agent reads execution logs to attribute failures to agents, and an Updating Agent modifies the prompts and graph structure. This is the only system in the survey where the harness topology is structurally modified in response to execution feedback.

Runtime self-reorganization is SEW [312]’s approach: the system generates and mutates entire workflow specifications using Direct Evolution (DE) and Hyper Evolution (HE) operators applied to LLM-generated workflow descriptions in structured formats (BPMN, CoRE, Python, YAML). Rather than optimizing agent parameters, SEW [312] optimizes the workflow structure including the sequence of agent calls, the routing logic, and the feedback paths. The two canonical topologies it discovers (a linear chain and a feedback loop) emerge from optimization rather than being hand-designed. FlowReasoner [338] pushes this adaptive view further by training a query-level meta-agent that generates a tailored multi-agent system for each input problem under external execution feedback, making topology selection itself part of the deliberative inference process rather than a fixed system design.

Table 10: Representative MAS execution-feedback and convergence designs.

System	Harness Substrate	Topology	Execution Feedback	Convergence
<i>Pre-defined topology</i>				
AgentCoder [50]	Execution	Cyclic	Test pass/fail	Correctness (test-gated)
MAGE [339]	Execution (waveform)	Chain-cyclic	Checkpoint waveform	Score-based correctness
MapCoder [44]	Execution, implicit	Cyclic	Test pass/fail	Correctness
AutoSafeCoder [52]	Execution (static, fuzzer)	Cyclic	CWE warnings, crashes	Security convergence
QualityFlow [253]	Execution (real, imagined)	Gated cyclic	Pass/fail, imagined exec	Correctness (quality-gated)
CodeCoR [166]	Execution, implicit	Cyclic	Syntax, test pass/fail	Score-based soft correctness
MARCO [345]	Execution (performance)	2-node Cyclic	Time, memory, FLOPS	Performance, correctness
<i>Adaptive topology</i>				
SoA [212]	Execution, implicit gap	Hierarchical tree	Test pass/fail	Correctness (implicit fallback)
SEW [312]	Implicit	Evolution	Test pass/fail	Implicit
EvoMAC [328]	Execution	Text DAG	Compiler, execution logs	Correctness (fixed-iteration)
FlowReasoner [338]	Execution, implicit	Query workflow	Execution feedback	Objective-driven adaptive
Trae Agent [336]	Repository, execution	Search pipeline	Test, pruning signals	Score-/selection-based

4.2. Execution Feedback and Shared-Harness Synchronization

We discuss how a group of agents can exploit the executability of code, and how they maintain a consistent shared view of the program state. This dimension is the defining one for code-centric MAS: the shared harness is uniquely executable and produces objective oracle signals. We address two sub-questions: what types of execution feedback are used, and how is shared state synchronized across agents.

4.2.1. Execution Feedback Integration

Compiler and syntax feedback Compiler and syntax feedback catch structural errors before runtime and are used by many systems. ChatDev [330] feeds compiler errors from the testing phase back to the programmer, though only as one-off corrections within a single phase. L2MAC [344] runs syntax checks via its evaluator module $E(D)$ after every file write, treating them as blocking conditions that prevent the instruction pipeline from advancing.

Test pass/fail signals Test pass/fail signals are the most commonly used execution-feedback type. AgentCoder [50] centers its entire loop on whether independently generated test cases pass; the iteration terminates on full pass or at the 5-iteration budget. QualityFlow [253] introduces a notable variant: Imagined Execution, in which an LLM simulates the Python interpreter step-by-step and predicts test outcomes without actually running the code, achieving 98%+ precision and recall on MBPP while avoiding label leakage from visible test cases. The near-identical performance of Self-Collaboration [56]’s simulated LLM tester and its real-compiler ablation raises a provocative empirical question: when is actual execution necessary, and when can linguistic simulation of execution suffice?

Fuzzer crash traces Fuzzer crash traces represent a qualitatively different type of feedback: rather than a pass/fail outcome, they provide a concrete failing input. AutoSafeCoder [52] uses type-aware mutation

to generate crash-inducing input seeds and passes the crashing input plus exit code to the Coding Agent. This adversarial feedback is more informative than a generic failure signal because it localizes the bug to a specific input category.

Static analysis warnings Static analysis warnings provide feedback about code structure and security properties without execution. AutoSafeCoder [52] uses CWE-mapped static analysis against the MITRE vulnerability database, enabling the Static Analyzer Agent to suggest remediation strategies keyed to specific vulnerability classes.

Performance profiling results Performance profiling results are uniquely exploited by MACRO [345], which treats code optimization as the primary task rather than correctness. The Performance Evaluator Agent measures execution time, memory usage, and FLOPS, and MACRO [345] uniquely augments this with real-time web search to retrieve relevant optimization techniques from the research literature.

Fine-grained simulation feedback MAGE [339]’s distinctive contribution is the finest-grained execution feedback in the surveyed literature. Rather than reporting only whether a testbench passes or fails, the State Checkpoint mechanism records signal values at every clock edge and delivers to the Debug Agent a waveform window around the first failing clock cycle. This enables targeted repair at sub-test granularity.

4.2.2. Shared-Harness Synchronization

Sequential handoff is the most common synchronization mechanism: each agent receives the output of its predecessor and passes its own output to its successor. The program state exists only in the form of the most recent artifact in the pipeline. This is sufficient for simple linear pipelines but creates invisible state divergence in multi-agent settings where multiple agents modify the codebase in parallel or iteratively. It is also where the limits of code-mediated coordination become clear. Even when agents share executable artifacts, the harness still imposes information-theoretic constraints: channels have finite bandwidth, summaries introduce compression loss, logs become noisy, cached views go stale, and parallel branches raise unresolved questions of authority and consistency. Code provides a richer substrate for coordination, but it does not remove these distributed-systems constraints.

Shared blackboard Shared blackboard provides a globally accessible program state that all agents can read and update. L2MAC [344] implements this most cleanly: the file store D is an external, persistent structure that is never overwritten but extended and revised. The Control Unit manages all reads and writes, ensuring that each agent invocation receives a precisely controlled context window. MAGIS [332]’s repository evolution memory M is a persistent key-value store mapping file versions to LLM-generated summaries, updated incrementally via a specialized blackboard for repository-level reasoning. Self-Collaboration [56] is among the first systems to explicitly name and invoke the blackboard architecture, establishing a shared memory from which all three roles read and write.

Parallel branches with merge Parallel branches with merge arise when multiple agents modify independent components simultaneously, with their changes integrated at a later stage. MAGIS [332] instantiates one Developer per candidate file; each modifies its assigned file independently, and all changes are merged into

the final repository diff. HyperAgent [333] runs multiple Navigator and Editor instances in parallel via Redis queues, with results merged at the Planner level.

Structured context scheduling Structured context scheduling is the explicit management of what each agent sees and when. It is the primary innovation of L2MAC [344]. The Control Unit resets the context window between instruction steps, providing each new invocation with a targeted summary of prior progress (M_{rs}) rather than the full conversation history. When the context window approaches capacity, the Control Unit stores partial results to D and re-initializes with a compressed view, explicitly instructing the LLM which files to read or skip given the remaining context margin. This mechanism solves the context-window problem not by expanding the window but by carefully controlling its contents. MetaGPT [55] implements a lighter form of context scheduling via a publish-subscribe message pool: each agent subscribes only to the document types relevant to its role, receiving a filtered view of the shared state.

Hierarchical memory Hierarchical memory combines short-term working context with longer-term accumulated knowledge. ChatDev [330] explicitly separates short-term memory (full dialogue within a phase) from long-term memory (extracted solutions carried across phases). Cogito [346] implements hierarchical memory, drawing on neurobiological architecture: short-term memory for immediate task state, a long-term knowledge base for accumulated expertise, and growth units for evolving abstractions that improve over time. HyperAgent [333] uses a lightweight LLaMA-3.1-8B summarizer to condense execution logs before storing them in hierarchical memory, preventing context bloat.

Agent pool scaling Agent pool scaling addresses the context-management problem orthogonally: rather than managing what a single agent sees, it distributes the context load across more agents. SoA [212] is the canonical example: by spawning more agents as task complexity grows, each agent’s context remains bounded. This is a structural solution to the harness-state problem: instead of building a shared representation that all agents can query, SoA [212] partitions the task state across agents, each holding a bounded slice. The limitation is that global consistency is sacrificed: agents cannot reason about the full program, only their assigned sub-function.

Other QualityFlow [253]’s revert mechanism represents a synchronization pattern: the initial code artifact is never overwritten, enabling the system to roll back to a prior shared harness state if the debugging trajectory degrades quality. This is the only work among the surveyed system that explicitly manages state history rather than always moving forward.

4.3. Position: The Shared Code-Centric Harness Substrate

We propose a new position for the next generation of multi-agent intelligence: the shared code-centric harness substrate. This position is motivated by the central gap identified in the literature: the lack of formal, persistent representations of the shared code state that agents can query and update across iterations. We argue that building such a harness substrate is both feasible and necessary for achieving robust, scalable multi-agent intelligence.

Table 11: Representative MAS designs centered on shared program-state representation and synchronization.

System	Harness Substrate	Agent Roles	Execution Feedback	Convergence / Synchronization
L2MAC [344]	Blackboard, repository, execution	Plan, Synth, Verif (evaluator)	Syntax, test pass/fail	Correctness per instruction step
Cogito [346]	Blackboard (3-tier memory)	Neurobiological model	NA	Hierarchical memory synchronization
CleanAgent [341]	Execution (weak), implicit	Plan, Understand, Synth, Exec	Runtime errors	Correctness through execution success
Lingma SWE-GPT [340]	Repository, execution	Understand, Synth-Verif	Syntax, git apply, tests	Fixed-limit implicit convergence
SyncMind [347]	Repository, execution (formal S_k/B_k)	Synth-Understand, oracle Understand	Test pass/fail, runtime errors	Correctness, resource-constrained synchronization
BOAD [337]	Repository, execution	Orchestrator with specialized sub-agents	Test pass/fail, validation reward	Hierarchy discovery, coordination
CANDOR [342]	Execution (Java, JaCoCo)	Plan, Synth, Verif, Understand, Debate	Compiler, coverage, tests	Correctness, coverage, consensus

4.3.1. Shared Harness Representation

A foundational question for any MAS is: what is the substrate these agents inhabit? In code as agent harness, the natural answer is the shared program environment, namely the collection of artifacts, execution contexts, and quality signals that agents collectively act upon and that evolve as agents produce, revise, and evaluate code. We call this the shared harness substrate, and we distinguish four levels of formalization with which existing systems represent it.

Implicit / File-only Representation The most common and least formalized category treats the shared harness as simply the current code file or set of code files. Agents receive the latest code artifact as part of their input context and produce a modified or evaluated version. There is no persistent, queryable representation: the shared state is reconstructed implicitly at each agent invocation from the conversational history. This category encompasses many foundational systems: ChatDev [330], MetaGPT [55], FlowGen [335], MapCoder [44], CodeCoR [166], SEW [312], and CodePori [331]. While this representation is simple to implement, it entails a fundamental limitation: agents cannot reason about the shared substrate except through the narrow lens of their most recent context window. State divergence [347], in which an agent’s internal belief about the code state diverges from the true state, is invisible to the system and cannot be detected or corrected.

Repository-based Representation A richer class of systems represents the shared harness as a navigable repository: a file system with directory structure, inter-file dependency graphs, call hierarchies, and version history. This representation supports agents that reason about where in the codebase a change needs to be made, what other components depend on the changed function, and how the codebase has evolved over time. MAGIS [332] introduces a repository evolution memory that caches file-level summaries and incrementally updates them via git diff as files change across issue-resolution episodes. HyperAgent [333] provides agents with repository navigation tools (`get_tree_structure`, `go_to_definition`, `code_search`, `get_all_references`),

treating the repository as a structured knowledge base. Lingma SWE-GPT [340] compresses the repository view via abstract syntax tree (AST) skeletons, preserving function signatures and class definitions to enable efficient navigation. SyncMind [347] is the only work to formally define the repository substrate as a ground-truth state S_k and measure the divergence between S_k and an agent’s belief state B_k .

Execution-based Representation Execution-based representation is the most distinctive category for code generation. It has no direct parallel in general MAS and represents the shared substrate through execution behavior. The state is not what the code looks like but what the code does: whether it compiles, which tests it passes, what vulnerabilities a fuzzer uncovers, how fast it runs, and whether its runtime behavior matches its specification. This execution-based representation provides an objective oracle signal, a ground truth that is not subject to the hallucination or bias that affects purely linguistic agent evaluations. Systems that exploit this representation include AgentCoder [50], AutoSafeCoder [52], QualityFlow [253], MACRO [345], EvoMAC [328], CANDOR [342], and MAGE [339]. Notably, MAGE [339] achieves the finest-grained execution feedback in the literature, operating at clock-edge granularity via *State Checkpoint* waveform snapshots.

Blackboard / Shared-State Representation A fourth category introduces an explicit, globally accessible data structure that all agents can read from and write to (akin to the classical blackboard architecture in AI [348]). This shared state is the closest approximation in the literature to a formal harness substrate: it persists across agent invocations, can be queried and updated, and provides a consistent view of the program state to all agents. Self-Collaboration [56] is among the first systems to explicitly invoke the blackboard metaphor, establishing a shared memory from which all three roles (Analyst, Coder, Tester) read and write. L2MAC [344] implements the most principled blackboard in the literature: a persistent file store D with semantically meaningful paths, accessed through a Control Unit that explicitly manages which slice of state each agent invocation sees. GameGPT [210] uses a shared context buffer to reduce redundant information retransmission in multi-round game development. Cogito [346] draws on neurobiological architecture to implement a three-tier memory: short-term working state, long-term knowledge base, and growth units for evolving abstractions, as a structured harness representation.

The Central Gap The distribution of systems across these four categories reveals a striking pattern: the majority of the literature resides in the implicit/file-only category, lacking any formal model of the shared harness substrate. This is the central gap that motivates the code as agent harness framing. The program, uniquely among multi-agent domains, is an artifact that executes. It produces objective, non-linguistic signals that could in principle anchor a formal shared substrate. Yet most systems fail to exploit this property at the architectural level, instead relying on agents to reason about code quality through natural language alone.

4.3.2. Harness-State Convergence

Convergence determines when a multi-agent coding harness should stop iterating and accept its current program state as a satisfactory outcome. In many existing MAS, convergence is still defined implicitly, either by consensus among agents or by an external iteration budget. However, code as agent harness has a distinctive advantage: because the shared substrate is executable, convergence can be grounded in objective behavioral signals rather than in conversational agreement alone. We identify six convergence patterns, ranging from widely used test-gated and implicit convergence to less common security-, performance-, and consensus-based criteria.

Correctness convergence Correctness convergence (test-gated) is the most principled and widely used objective criterion: the system terminates successfully when all test cases pass. AgentCoder [50], L2MAC [344], SyncMind [347], and CANDOR [342] implement test-gated convergence. PairCoder [334] augments this with dead-end detection: if the same buggy code or feedback appears in the iteration history, the system switches to the next candidate plan rather than looping. FlowGen [335] uses test-gated convergence but on LLM-generated tests rather than ground-truth tests, introducing a potential quality concern: a system can converge on code that passes its own biased tests but fails on external evaluation.

Security convergence Security convergence is uniquely implemented by AutoSafeCoder [52]: the system terminates successfully when no CWE vulnerabilities are flagged by static analysis and no crashes are induced by the fuzzer. This multi-criteria convergence is a strong argument for the execution-based harness framing. Both convergence criteria are grounded in objective program behavior, not agent opinions.

Performance convergence Performance convergence is the focus of MACRO [345]: the optimization loop terminates when user-defined runtime and memory thresholds are satisfied, as measured by the Performance Evaluator against actual execution benchmarks. This is the only system that treats performance as the primary convergence criterion rather than correctness.

Score-based convergence Score-based convergence uses quantitative quality scores computed by agents evaluating intermediate outputs to determine when to stop. MAGE [339] ranks candidate programs by their simulation mismatch score $s(r) = 1 - m(r)/tc(r)$ and continues iterating until the maximum score reaches 1.0. CodeCoR [166] uses a four-criteria binary score (clarity, relevance, conciseness, context) to prune intermediate outputs at each agent stage and selects the highest-ranked code in its Ranked Code Set as the final output. It sets a soft correctness convergence that submits the best available result rather than waiting for a perfect solution. Trae Agent [336] introduces a closely related search-and-selection view at repository scale: it formulates issue resolution as an optimal solution search problem and uses modular generation, pruning, and selection agents to navigate a large ensemble space of candidate patches. In this setting, convergence is not only a matter of repeated repair, but also of ranking, filtering, and selecting among competing solutions under repository-aware evidence.

Consensus convergence Consensus convergence aggregates judgments from multiple reviewer agents. CANDOR [342] implements majority voting among three Panelists on oracle correctness. MAGIS [332] uses LLM-judgment from the QA Engineer as the acceptance signal, though this is a single-agent consensus rather than a multi-agent vote. QualityFlow [253] uses its Code Quality Checker as the single gating signal. It is an efficient design where the quality checker serves as both a convergence oracle and the system controller, enabling early exit (75–84% of problems converge after the first generator call).

Implicit convergence Pipeline termination after a fixed number of stages or iterations with no objective quality criterion is the most prevalent convergence pattern in the literature and represents the most significant gap in the field. ChatDev [330] terminates after a fixed number of phases, or when two consecutive rounds produce identical code, or after 10 rounds, none of which is an objective quality signal. MetaGPT [55] terminates after completing the fixed SOP stages. Self-Collaboration [56] falls back to implicit convergence after $n = 4$ iterations if the tester never approves. EvoMAC [328] runs a fixed K iterations of the textual

backpropagation loop. The prevalence of implicit convergence is a direct consequence of the lack of formal shared substrates: without an objective representation of the program state, systems have no principled criterion for convergence.

4.4. Patterns and Trends

Across systems, differences in role specialization, shared-state representation, execution grounding, and workflow topology are not independent engineering choices; they interact to determine how reliably a group of agents can maintain coherence over long-horizon coding tasks. In this subsection, we distill the main trends that emerge from the surveyed systems, highlighting both the common structural bottlenecks of current systems and the design principles that point toward more robust shared harnesses.

The implicit-harness-state constraint The majority of surveyed systems (ChatDev [330], MetaGPT [55], FlowGen [335], CodePori [331], SEW [312], MapCoder [44], CodeCoR [166]) operate without explicit representations of the shared code harness. These systems rely on agents to reconstruct state implicitly from conversational history at each invocation. This design choice works for function-level tasks where the program state is simple and does not fragment across agents. However, this implicit approach creates a fundamental vulnerability: without a formal shared substrate, agents cannot reliably detect when their internal understanding diverges from the true program state [347]. From the code as agent harness perspective, the reliance on implicit state representations is the technical root of system brittleness rather than a scalability convenience.

Code-mediated channels do not eliminate coordination bottlenecks The shift from free-form dialogue to code-mediated coordination is a genuine architectural advance, but it should not be overstated. Files, APIs, diffs, tests, logs, schemas, blackboards, and workflow states are all partial channels through which task state is encoded, transmitted, and reconstructed. Each channel trades off fidelity, latency, and scope: tests compress semantics into pass/fail, summaries save context at the cost of detail, logs are grounded but noisy, and shared blackboards improve persistence while creating authority and consistency problems. The central design question is therefore not merely whether code is present, but which artifacts are authoritative, how they are compressed, and how conflicts across channels are resolved.

Execution feedback as the bridge between linguistic and formal reasoning The deepest divide in the literature is between systems that use execution as ground truth and those that rely on linguistic model judgments. Systems that ground shared state in execution (AgentCoder [50], AutoSafeCoder [52], QualityFlow [253], EvoMAC [328], MAGE [339]) have access to objective oracle signals, signals that cannot hallucinate. Yet a surprising finding complicates this picture: Self-Collaboration [56] and QualityFlow [253] demonstrate that LLM-simulated execution can achieve 98%+ precision and recall in predicting actual outcomes without running code. This suggests that execution feedback’s value is not uniform across all failure modes. It excels at detecting the corner cases that linguistic simulation structurally cannot imagine (runtime crashes, resource exhaustion, boundary condition errors, performance regressions), but for many correctable bugs, simulated reasoning may suffice. A mature harness would integrate both: using linguistic reasoning as the fast path and delegating to execution as the verification oracle only for the failure modes that require it.

Two complementary representations of the shared harness The surveyed systems reveals two conceptually orthogonal views: repository-based representation (structure: what functions call what, where does data flow, what are the dependencies) and execution-based representation (behavior: what does the code do when run, how does state evolve at runtime, what emergent failures occur under different inputs). MAGIS [332] and HyperAgent [333] operate primarily in the repository view, enabling agents to reason about codebase architecture. AgentCoder [50] and MAGE [339] operate primarily in the execution view, grounding shared state in runtime signals. Yet none of the surveyed systems fully unifies both views into a single harness substrate where agents can reason across both the static structure of code and its dynamic behavior. The deepest harness would integrate these two perspectives, answering questions like “which components are slow” (requires both call graphs and profiling data) or “does this refactoring break APIs that external code depends on” (requires both static analysis and dynamic testing).

Topology complexity inversely correlates with harness-state formality Systems with explicit, formal shared substrates use simpler topologies, while systems lacking formal shared state employ increasingly complex topology patterns as a structural workaround. L2MAC [344], which has the clearest formal harness substrate (a persistent file store with explicit context scheduling), uses a simple sequential chain with sophisticated state management. By contrast, implicit-state systems like EvoMAC [328] and SEW [312] develop elaborate adaptive topologies (dynamic DAGs, workflow mutation, agent pool scaling) that attempt to optimize the collaboration structure in the absence of a principled shared representation. This suggests that topology complexity is partially a symptom: when the substrate is formally represented and queryable, agents can coordinate through simple, transparent protocols. When the substrate is implicit, agents require richer interaction patterns to compensate for missing state information.

Context management is the tax of implicit shared state A striking pattern is that many systems have developed sophisticated context-management mechanisms precisely because they lack a formal shared substrate. L2MAC [344]’s Control Unit, MetaGPT [55]’s publish-subscribe pool, SoA [212]’s agent-pool scaling, and Cogito [346]’s three-tier memory are all responses to the same underlying problem: how to give agents a coherent view of a code harness that is too large to fit in any one context window. A mature harness substrate could unify these disparate solutions by providing a principled, queryable representation of task state that agents access on demand, rather than forcing the system to carefully manage what each agent sees at every step.

Agent specialization increases the criticality of shared state metrics As agent role diversity increases, from basic coder-tester pairs to systems with Architect, Manager, Navigator, Executor, and Verifier roles, the need for a unified shared substrate becomes urgent. Without shared understanding of code state, the Planning Agent may decompose tasks based on an outdated codebase snapshot, the Execution Agent may run tests against a different version than the Synthesis Agent intended, and the Verification Agent’s feedback may misfire. EvoMAC [328] addresses this through its Gradient and Updating agents that explicitly monitor failure attribution at the MAS level. SyncMind [347] formalizes the problem as agent belief divergence $|B_k - S_k|$, proposing explicit synchronization protocols. The proliferation of agent roles is thus not merely an engineering choice. It is a forcing function for developing more mature shared harnesses. Multi-agent systems with rich role repertoires cannot function robustly without them.